

Design and Implementation of a Security Filter for Detecting Malicious File Uploads in Web Applications

*A project prepared to obtain a Bachelor's degree in Artificial Intelligence Engineering,
specializing in Intelligent Information Security Systems Engineering — Senior Project II*

Prepared by:

Ahmad Khalluof
Talal Alhelou

Supervised by:

Dr. Wasim Juneidi
Eng. Mohammed Yamen Hallak

August 2026

Abstract

File upload is one of the most commonly exposed and most frequently abused features of modern web applications. A single endpoint that accepts user-supplied files must simultaneously defend against a wide spectrum of attacks — from trivial extension spoofing, through polyglot files that are at once a valid image and a valid script, to malicious documents whose payloads are concealed behind layered stream compression. Conventional upload validators apply a single check — an extension whitelist, a MIME sniff, or one signature scan — and are therefore defeated by any input crafted to satisfy that one check while smuggling an executable payload past it.

This project presents SecureUploader, a defense-in-depth pipeline of nine ordered inspection layers that transforms an untrusted uploaded file into either a rejected verdict accompanied by a precise per-layer trace, or an accepted-but-neutralized artifact. Beyond detection, SecureUploader treats neutralization — the deterministic rewriting of a file to strip an embedded payload while preserving its legitimate content — as a first-class outcome and as a second evaluation metric alongside detection. The pipeline hardens several evasion classes that defeat single-check validators: nested-filter PDF compression, image-carried scripts, and active SVG content.

SecureUploader was evaluated on a 10,012-file corpus drawn from CIC-Evasive-PDFMal2022, achieving 95.48% recall, 93.05% precision, 93.54% accuracy, and a 94.25% F1 score, and was further stress-tested against a hand-built adversarial corpus of real polyglots and evasive fixtures, compared side-by-side against five reference validators. A dedicated study on a pixel-domain steganography dataset showed that a deterministic least-significant-bit clearing step neutralizes every one of the sixty payload-carrying images in the tested sample, while the fifteen clean control images were accepted and the transformation remained visually imperceptible (peak signal-to-noise ratio no lower than 49.35 dB, mean 52.99 dB). Six self-bypass vulnerabilities discovered during development were diagnosed and fixed, and every verdict is accompanied by a reproducible, per-layer evidence trail — providing the transparency that opaque machine-learning detectors lack.

Keywords: file-upload security, defense in depth, polyglot files, evasion resistance, content neutralization, magic-byte validation, YARA signature scanning, PDF FlateDecode inflation, SVG sanitization, empirical evaluation.

إهداء

إلى والديّ الكريمين، اللذين كانا سندي ودعمي في كل خطوة...
إلى أساتذتي الكرام، على جهودهم وتوجيههم طوال مسيرة هذا البحث...
إلى كل من أسهم في إثراء مسيرتنا العلمية بعلمه أو تشجيعه...

طلال الحلو — أحمد خلوف

ملخص البحث

يُعدّ رفع الملفات (File Upload) من أكثر الميزات انتشاراً في تطبيقات الويب الحديثة، وفي الوقت نفسه من أكثرها استغلالاً من قبل المهاجمين. فنقطة النهاية (Endpoint) التي تقبل ملفات المستخدم يجب أن تتصدّى في آنٍ واحد لطيفٍ واسع من الهجمات؛ بدءاً من انتحال الامتداد البسيط، مروراً بملفات الـ Polyglot، وصولاً إلى المستندات الخبيثة التي تُخفي حملتها خلف طبقاتٍ متتالية من الضغط.

يقدم هذا المشروع أداة SecureUploader، وهي خط دفاعٍ متعدد الطبقات مؤلّف من تسع طبقات فحص مرتّبة، يحوّل الملف غير الموثوق إلى إمّا رفضٍ مصحوبٍ بأثرٍ دقيق لكل طبقة، أو قبولٍ بعد تحييد الحمولة (Neutralization). جرى تقييم الأداة على 10012 ملفاً مأخوذة من CIC-Evasive-PDFMal2022، محقّقةً Recall 95.48% و Precision 93.05% و Accuracy 93.54% و F1 94.25%، ثم اختُبرت على مجموعةٍ خصامية مبنية يدوياً. كما أظهرت دراسةً مخصّصة على مجموعة صور الإخفاء بمجال البكسل (LSB) أن خطوة تصفير البت الأقل أهمية تُبطل الحمولات في الصور السّتين الحاملة للحمولة من العيّنة المُختبرة، مع قبول الصور النظيفة الخمس عشرة الضابطة، وبقاء التشويه غير محسوس بصرياً (PSNR لا يقلّ عن 49.35 dB بمتوسّط 52.99 dB). وقد عولجت ستّ ثغراتٍ للتجاوز الذاتي اكتُشفت أثناء التطوير.

الكلمات المفتاحية: أمن رفع الملفات، الدفاع متعدد الطبقات، ملفات Polyglot، مقاومة التحايل، تحييد المحتوى، YARA، التقييم التجريبي.

Table of Contents

Abstract	2
إهداء	3
ملخص البحث	4
Table of Contents	5
List of Figures	10
List of Tables	11
List of Abbreviations and Technical Terms.....	12
Chapter One — Introduction	13
1.1 Introduction.....	14
1.2 Problem Statement.....	15
1.3 Research Objective	15
1.4 Research Questions.....	16
1.5 Scope and Assumptions	16
1.6 Practical Applications	17
1.6.1 Hardening Web-Application Upload Endpoints.....	17
1.6.2 Supporting Security Operations and Triage.....	17
1.6.3 Integration into Continuous-Delivery Pipelines	17
1.6.4 A Foundation for Research and Coursework.....	18
1.7 Challenges.....	18
1.8 Literature Review.....	19
1.8.1 Published Upload-Checking Systems (FileUploadChecker).....	19
1.8.2 Evasion Taxonomies for File Upload (FUEL / Fuxploidier-NG).....	19
1.8.3 Empirical Study of Polyglot Files (Koch et al.).....	20
1.8.4 Machine-Learning Detection of Malicious Documents.....	20
1.8.5 Re-encoding and Round-Trip Sanitization	20
1.8.6 The Research Gap	21
1.9 Structure of the Report.....	21
Chapter Two — Theoretical Background.....	22
2.1 Introduction.....	23
2.2 File Formats and How Parsers Read Them.....	23
2.2.1 JPEG Structure.....	24
2.2.2 PNG Structure.....	24
2.2.3 PDF Object Model	24

2.2.4 SVG and the XML Model.....	24
2.3 The File-Upload Attack Surface	25
2.3.1 HTTP Upload Mechanics	25
2.3.2 Storage, Processing, and Serving Paths	26
2.4 A Taxonomy of File-Upload Attacks	26
2.4.1 Extension Spoofing.....	27
2.4.2 MIME / Content-Type Spoofing.....	27
2.4.3 Web Shells	28
2.4.4 Polyglot Uploads.....	28
2.4.5 Stored Cross-Site Scripting via Uploads.....	28
2.4.6 XXE and SSRF via XML-Based Formats	28
2.4.7 Compression and Decompression Attacks.....	28
2.5 Polyglot Files in Depth	29
2.5.1 A Worked Polyglot Example	30
2.6 Nested-Filter and Compression Concealment	30
2.6.1 Decompression Bombs and Expansion Ratios	31
2.7 Active Content in Vector Formats	32
2.8 Inspection Building Blocks.....	32
2.8.1 Extension Whitelisting.....	32
2.8.2 Magic-Byte / Signature Validation	32
2.8.3 Header–Content Matching	33
2.8.4 YARA Signature Scanning.....	33
2.8.5 Content Re-encoding and Sanitization.....	34
2.9 Neutralization as a Distinct Outcome	34
2.10 Mapping Evasion Techniques to Defenses	34
2.11 Comparison of Detection Strategies	35
2.12 Chapter Summary	36
Chapter Three — Methodology and Environment	37
3.1 Introduction.....	38
3.2 Research Methodology	38
3.3 Development Platform and Language	38
3.4 Libraries, Tools, and Environment	39
3.5 Threat Model.....	39
3.6 Evaluation Datasets and Corpus Construction.....	40

3.6.1 The Public Corpus (CIC-Evasive-PDFMal2022)	41
3.6.2 Real Polyglots and the Download Route	41
3.6.3 Benign Corpus and Inert Fixtures	41
3.7 Containment and the Handling of Real Malware	42
3.8 Accepted File Types and Threat Scope	42
3.9 Testing Environment and Host Controls.....	43
3.9.1 Determinism and Reproducibility.....	43
3.10 Evaluation Metrics	44
3.11 Chapter Summary	44
Chapter Four — System Design and Implementation	46
4.1 Introduction.....	47
4.2 Design Principles	47
4.3 High-Level Architecture	47
4.4 The IScanLayer Abstraction and Orchestration.....	49
4.4.1 Extending the Pipeline with a New Layer	52
4.5 Structural Layers (1–4)	52
4.5.1 ExtensionWhitelist (Layer 1).....	52
4.5.2 MagicBytes (Layer 2)	52
4.5.3 HeaderComponentMatching (Layer 3)	53
4.5.4 DoubleExtension (Layer 4).....	54
4.5.5 Error Handling and Fail-Closed Behavior	54
4.6 Detection Layers (5, 5b, 5c).....	55
4.6.1 YARA Signature Scanning (Layer 5)	55
4.6.2 PDF FlateDecode Recursive Inflation (Layer 5b)	55
4.6.3 Embedded-Link Inspection (Layer 5c)	57
4.7 Transformation Layers (6, 7)	58
4.7.1 Image Re-encoding (Layer 6)	58
4.7.2 SVG Sanitization (Layer 7)	61
4.8 Caching Subsystem.....	63
4.9 Configuration Model.....	65
4.10 Security-Headers Middleware	66
4.11 Verdicts and the Evidence Trail.....	67
4.11.1 Reading a Trace	68
4.12 The Instrument-Panel Frontend	68

4.13 Worked Example: A File's Journey Through the Pipeline	68
4.14 Chapter Summary	69
Chapter Five — Experimental Results	70
5.1 Introduction.....	71
5.2 Evaluation Setup	71
5.3 Aggregate Results	71
5.3.1 Confusion Matrix and Metrics	71
5.3.2 Interpretation.....	73
5.3.3 False-Positive Analysis.....	73
5.3.4 False-Negative Analysis	74
5.4 Adversarial-Corpus Comparison	74
5.5 Per-Sample Analysis	77
5.5.1 EICAR Behind a Single FlateDecode.....	78
5.5.2 EICAR Behind a Double FlateDecode	78
5.5.3 Appended-Payload Neutralization	78
5.5.4 Cache Behavior	79
5.5.5 PNG Text-Chunk Parasite.....	79
5.5.6 WebP with Appended Bytes	79
5.5.7 End-to-End Walkthrough of a Neutralized Upload	80
5.6 Neutralization of Pixel-Domain Steganography (LSB).....	80
5.6.1 Dataset and Metric	81
Table 17: The pixel-domain steganography dataset and the tested sample.	82
5.6.2 Results.....	82
Table 18: Per-class neutralization of LSB payloads on the tested sample.	83
5.6.3 Technique and Scope	83
5.7 Timing and Performance.....	84
5.8 Contextual Comparison with Machine-Learning Detectors	85
5.9 Comparison with Related Systems	86
5.10 Chapter Summary	86
Chapter Six — Discussion	87
6.1 Introduction.....	88
6.2 Why Layering Resists Evasion	88
6.3 The Value of Ordering.....	88
6.4 Self-Bypass Findings and Fixes.....	88

6.4.1 Nested-Filter Evasion.....	89
6.4.2 The Antivirus / Scanner Race	89
6.4.3 Stale-Cache Masking	90
6.5 Detection versus Neutralization.....	90
6.6 Transparency and Auditability.....	90
6.7 Qualitative Comparison with Prior Approaches	91
6.8 Limitations	92
6.9 Threats to Validity	94
6.10 Deployment Considerations.....	94
6.11 Ethical Considerations	95
6.12 Chapter Summary	95
Chapter Seven — Conclusion and Future Work.....	97
7.1 Summary of the Work.....	98
7.2 Revisiting the Research Questions.....	98
7.3 Contributions.....	99
7.4 Immediate Next Steps	99
7.5 Longer-Term Directions	100
7.6 Reflections and Lessons Learned.....	100
7.7 Concluding Remarks.....	101
References	102

List of Figures

<i>A taxonomy of file-upload attacks</i>	27
<i>Anatomy of a polyglot file</i>	29
<i>Recursive inflation of a nested FlateDecode stream</i>	31
<i>Threat model and trust boundary</i>	40
<i>SecureUploader pipeline architecture</i>	48
<i>Pipeline orchestration via IScanLayer</i>	49
<i>CachedScanLayer three-part cache key</i>	64
<i>Confusion matrix and performance metrics</i>	72
<i>Aggregate detection metrics</i>	72
<i>False-positive rate before and after severity-gating</i>	74
<i>Detection / neutralization matrix, with type-or-size refusals marked separately</i>	76
<i>Adversarial-corpus detection rate on the commonly accepted fixtures</i>	77
<i>Neutralization by image re-encoding</i>	79
<i>Pixel-domain (LSB) neutralization before and after LSB clearing (n = 60 payload-carrying; 15 clean controls accepted)</i>	83
<i>Relative per-layer processing cost</i>	85
<i>Qualitative comparison vs prior approaches</i>	91

List of Tables

The scope of this project: what is addressed and what is deliberately excluded	17
Principal classes of file-upload attacks and the property each abuses.	27
Polyglot construction taxonomy (after Mitra) and the layer that defeats each class.	29
Mapping of each evasion technique to the pipeline layer(s) that defeat it.....	35
Comparison of the core file-inspection strategies used in the pipeline.	35
Development platform, language, and key libraries.....	39
Threat-model assumptions: attacker capabilities, goals, and what is out of scope.....	40
Evaluation datasets and their roles in the corpus.....	40
Accepted file types and their in-scope threat classes.....	42
Definitions of the evaluation metrics used in Chapter Five.	44
The nine inspection layers, their category, and their responsibility.	48
Categories of YARA rules in the signature-scanning layer, with their counts and declared severities.....	55
SVG constructs handled by the sanitization layer, the risk each poses, and the resulting verdict.....	62
Runtime configuration sections exposed by the pipeline.....	66
The six validators, the logic each reproduces, and the types and sizes each will consider at all.	75
Per-fixture verdict of SecureUploader on representative adversarial samples.	77
The pixel-domain steganography dataset and the tested sample.	82
Per-class neutralization of LSB payloads on the tested sample.....	83
Contextual comparison with machine-learning detectors on CIC-Evasive-PDFMal2022...	85
The six self-bypass vulnerabilities found during development and their fixes.	89
Threats to validity and the mitigation applied to each.....	94

List of Abbreviations and Technical Terms

Abbreviation	Full Term
API	Application Programming Interface
AV	Antivirus
CIC	Canadian Institute for Cybersecurity
CWE	Common Weakness Enumeration
CVE	Common Vulnerabilities and Exposures
DoS	Denial of Service
DVWA	Damn Vulnerable Web Application
EICAR	European Institute for Computer Antivirus Research (test file)
FN	False Negative
FP	False Positive
FPR	False-Positive Rate
HTTP	Hypertext Transfer Protocol
MIME	Multipurpose Internet Mail Extensions
PDF	Portable Document Format
PE	Portable Executable
RCE	Remote Code Execution
RTP	Real-Time Protection
SHA-256	Secure Hash Algorithm, 256-bit
SSRF	Server-Side Request Forgery
SVG	Scalable Vector Graphics
TN	True Negative
TP	True Positive
XSS	Cross-Site Scripting
XXE	XML External Entity
YARA	Pattern-matching engine for malware classification

Chapter One — Introduction

1.1 Introduction

File upload is a near-universal feature of modern web applications. Content-management systems, e-learning portals, ticketing systems, medical and financial platforms, and social networks all expose at least one endpoint that accepts a file from an untrusted user and then stores, processes, or serves it. Each of these three actions turns a property of the uploaded bytes into a capability the attacker may be able to exploit, which is what makes file upload one of the largest and most consequential attack surfaces on the web.

Consider a concrete scenario. A photo-sharing site allows members to upload a profile picture, checking only that the filename ends in an image extension. An attacker uploads a file named `avatar.php.jpg` whose bytes begin with a valid image header but end with a small server-side script. If the server stores the file under a web-executable path and a later request reaches it as a script, the attacker has achieved remote code execution on a trusted host — from nothing more than a profile picture. Variations of this pattern, catalogued under CWE-434 (Unrestricted Upload of File with Dangerous Type), have appeared repeatedly in real systems for two decades.

The core difficulty is that a file is not what its name says it is. An attacker controls every byte of the uploaded content and every character of its filename, and can therefore craft an input that passes a naive check while behaving maliciously once stored. The simplest form is extension spoofing — renaming a payload to end in an allowed extension. More advanced techniques defeat deeper checks: polyglot files are simultaneously valid instances of two formats, so that the same bytes are read as a harmless image by one parser and as executable script by another; malicious documents conceal their payload behind several layers of stream compression so that a single-pass scanner never sees the decoded content; and vector formats such as SVG carry active script that executes in the victim's browser when the file is served back.

Conventional upload validation applies a single check in isolation — an extension whitelist, a MIME-type sniff, or one signature scan — and treats a pass on that check as sufficient. Because each check reasons about only one property of the file, any input engineered to satisfy that one property is accepted regardless of what else it contains. The consequence is a long history of documented bypasses in which files that are individually accepted by every deployed check nonetheless carry an executable payload.

This project addresses that gap with SecureUploader, a defense-in-depth pipeline that inspects an uploaded file through nine ordered layers, each reasoning about a different property, so that defeating the pipeline requires defeating all applicable layers at once rather than any single one. Where a file cannot be cleanly accepted or rejected on structural grounds alone, SecureUploader introduces a second option — neutralization — that deterministically rewrites the file to remove any embedded payload while preserving its legitimate content, converting an ambiguous input into a demonstrably safe one.

1.2 Problem Statement

The problem this project confronts can be stated precisely. Given an uploaded file consisting of attacker-controlled bytes and an attacker-controlled filename, decide from the bytes alone whether the file is safe to store and serve, and do so in a way that cannot be defeated by an input engineered to satisfy any single check. A validator that reasons about one property — extension, MIME type, or a single signature scan — fails this requirement by construction, because the attacker can satisfy that one property while concealing a payload elsewhere in the file. What is required instead is a validator whose acceptance depends on multiple independent properties simultaneously, together with a mechanism for handling files that are neither clearly safe nor clearly malicious without simply discarding legitimate uploads.

1.3 Research Objective

The objective of this project is to design, implement, and empirically evaluate a file-upload security pipeline that resists the evasion techniques which defeat single-check validators, and to establish neutralization as a measurable outcome alongside detection. Concretely, the project pursues the following goals:

- **Layered detection.** Build a pipeline of independent inspection layers — extension whitelisting, magic-byte validation, header-content matching, double-extension detection, YARA signature scanning, recursive PDF stream inflation, embedded-link inspection, image re-encoding, and SVG sanitization — that must all be satisfied for a file to be accepted.
- **Evasion resistance.** Explicitly harden the pipeline against nested-filter PDF compression, image-carried scripts, and active SVG content, and validate that hardening on crafted adversarial fixtures rather than on benign or randomly-sampled malicious files alone.

- **Neutralization as a second metric.** Treat the deterministic rewriting of a file to strip a payload while preserving legitimate content as a first-class verdict, and measure it alongside detection so that value invisible to a detection-only account is captured.
- **Transparency.** Attach to every verdict a reproducible, per-layer evidence trail identifying exactly which layer acted and why, in contrast to opaque machine-learning detectors whose decisions cannot be audited.
- **Empirical evaluation.** Quantify detection performance on a large public corpus and compare the pipeline, layer by layer, against a set of reference validators on a hand-built adversarial corpus.

1.4 Research Questions

The objectives above translate into five research questions that structure the evaluation in the later chapters:

- **RQ1.** Can an ordered pipeline of independent layers resist evasion techniques — polyglots, nested-filter concealment, and active SVG content — that defeat single-check validators?
- **RQ2.** How does the pipeline perform, in standard detection metrics, on a large and realistic corpus of malicious and benign documents?
- **RQ3.** How does the pipeline compare, fixture by fixture, against representative reference validators on inputs constructed specifically to evade validation?
- **RQ4.** What measurable value does neutralization add beyond binary accept/reject detection, and what are its limits?
- **RQ5.** Does designing each layer with its own bypass in mind reveal weaknesses in the pipeline itself, and can those weaknesses be fixed?

1.5 Scope and Assumptions

The project is deliberately scoped to keep the threat tractable and the evaluation meaningful. The pipeline accepts a narrow set of image and document types and inspects raw bytes without any learned model; formats whose semantics broaden the attack surface, and defenses that require training or full dynamic sandboxing, are out of scope. The boundaries are summarized below and revisited when the threat model is defined in Chapter Three.

In scope	Out of scope
Raster (JPEG, PNG, WebP) and vector (SVG) images, and PDF documents	Office documents, archives, and executable formats
Byte-level inspection with no training phase	Machine-learning training and feature extraction
Neutralization by re-encoding and sanitization	Full dynamic sandboxing / detonation
Single-file uploads at a public endpoint	Recursive expansion of container/archive uploads

Table 1: The scope of this project: what is addressed and what is deliberately excluded

Two assumptions bound the work. First, the analysis host and the pipeline itself are trusted; the attacker acts only through the upload endpoint. Second, the accepted file types are almost entirely non-executable on the host, which is what makes controlled testing with real malicious samples defensible under the host controls described in Chapter Three.

1.6 Practical Applications

The pipeline is designed as a reusable component that can be embedded wherever untrusted files enter a system. Its practical applications include the following.

1.6.1 Hardening Web-Application Upload Endpoints

SecureUploader can sit directly in front of any upload handler, rejecting or neutralizing malicious files before they are stored or served. Because it reasons about file content rather than trusting the filename or the client-supplied content type, it protects endpoints that would otherwise be bypassed by extension spoofing, MIME confusion, or polyglot inputs. Its narrow accepted-type set makes it a natural fit for the common case of an application that only ever needs to accept user images and documents.

1.6.2 Supporting Security Operations and Triage

The per-layer evidence trail produced for every file gives a security analyst an immediate, human-readable explanation of why a given upload was blocked or rewritten — which layer acted, on what evidence, and how long each stage took. This shortens triage, provides defensible documentation for each decision, and makes it possible to reproduce a verdict exactly rather than trusting an opaque score.

1.6.3 Integration into Continuous-Delivery Pipelines

Exposed as a service with a machine-readable verdict, the pipeline can gate artifact and asset uploads inside automated build and deployment workflows, preventing evasive files from entering a trusted software supply chain. Because it is training-free and deterministic, the same input always yields the same verdict, which is a prerequisite for use inside a reproducible build.

1.6.4 A Foundation for Research and Coursework

Because each layer is independent and its contribution is measurable in isolation, the pipeline serves as a practical platform for studying evasion techniques, ablating individual defenses, and exploring the trade-off between detection and neutralization. Its transparent design makes it suitable as a teaching artifact in a security curriculum.

1.7 Challenges

Building an upload pipeline that is both safe and usable raises several tensions that shaped the design of SecureUploader:

- **Coverage versus false positives.** A stricter policy blocks more attacks but also rejects more legitimate files. The pipeline must maximize detection while keeping the false-positive rate on genuine files low enough to be deployable, and must be able to explain and tune the causes of any false rejection.
- **Evasion by construction.** Adversaries craft inputs specifically to satisfy each known check, so every layer must be designed with its own bypass in mind and validated against inputs built to defeat it, not merely against representative samples.
- **Nested and recursive concealment.** Payloads hidden behind several layers of compression are invisible to a single-pass scanner; inflation must be recursive and bounded, expanding concealed content to a configurable depth without exposing the pipeline to decompression-bomb denial of service.
- **Neutralization without data loss.** Rewriting a file to strip a payload must be deterministic and must preserve the file's legitimate, visible content, so that a neutralized file remains useful to the end user rather than being silently corrupted.
- **Interaction with the host environment.** Security tooling on the analysis host can interfere with detection — for example by quarantining a sample before the pipeline can inspect it —

so the evaluation environment itself must be controlled, documented, and reasoned about as part of the methodology.

1.8 Literature Review

This section surveys the prior work most directly related to SecureUploader: published upload-checking systems, evasion taxonomies, empirical studies of polyglot files, machine-learning detectors for malicious documents, and the re-encoding defenses on which the neutralization layers build. Each body of work is examined for what it contributes and where it stops, and the section closes by stating the research gap that SecureUploader addresses.

1.8.1 Published Upload-Checking Systems (FileUploadChecker)

The closest published analog to this work is FileUploadChecker (ARES 2022), a server-side component that inspects uploaded files against a set of format-specific rules before they are accepted. It validates that a file conforms to the structure its type implies and rejects files that fail those checks, and it is the most direct point of comparison for SecureUploader.

Two differences in framing separate the present work from FileUploadChecker. First, SecureUploader adds explicit evasion-resistance hardening for nested-filter and polyglot inputs — recursive inflation and re-encoding — rather than validating format conformance alone. Second, it introduces neutralization as an accepted outcome, so that an ambiguous but potentially legitimate file can be rewritten into an inert form instead of being rejected outright. FileUploadChecker is therefore treated as the primary baseline against which the contribution is positioned.

1.8.2 Evasion Taxonomies for File Upload (FUEL / Fuxploider-NG)

FUEL (DIMVA 2024) provides a systematic taxonomy of file-upload evasion scenarios together with an automated tool for exercising them against a target. Its value is descriptive: it enumerates the ways an upload check can be bypassed and organizes them into scenario categories.

In this project FUEL is used as a taxonomic reference rather than re-implemented as a comparator. Its scenario categories anchor the coverage map presented in Chapter Two, which states, for each evasion class, the pipeline layer intended to defeat it. Using FUEL this way keeps the evaluation grounded in an external taxonomy rather than one invented for the occasion.

1.8.3 Empirical Study of Polyglot Files (Koch et al.)

Koch et al. (ACM WWW 2025) present a large-scale study of the abuse and detection of polyglot files, including a catalog of real-world polyglot samples together with their cryptographic hashes. The study documents how frequently polyglots occur in the wild and how poorly common tools detect them.

That catalog supplies the sample identifiers and the construction taxonomy on which this project's adversarial corpus is built, with real samples retrieved by hash through a single sanctioned route as availability permits. It also provides external support for the neutralization framing and for the rationale behind the image re-encoding layer, which destroys the secondary interpretation of a polyglot by rebuilding the file from decoded pixels — a defense the study's findings motivate directly.

1.8.4 Machine-Learning Detection of Malicious Documents

A substantial body of work applies machine learning to malicious-document detection. Representative studies on the CIC-Evasive-PDFMal2022 dataset report accuracies above 98% using classifiers trained on pre-extracted feature vectors — for example structural and metadata features of PDF documents fed to ensemble or stacking learners.

These results are strong but are not directly comparable to SecureUploader. They operate on curated feature tables with dedicated training and test splits, whereas SecureUploader operates on raw file bytes with no training phase and produces an auditable per-layer decision rather than a learned score. The comparison in this report is therefore contextual — situating a transparent, training-free, byte-level pipeline against opaque, trained, feature-level classifiers — rather than a head-to-head benchmark on the same operating regime.

1.8.5 Re-encoding and Round-Trip Sanitization

Prior work on image re-encoding as a sanitization defense (ICDR 2023) demonstrates that decoding an image to its pixel representation and re-encoding it destroys most appended or embedded payloads, because only the pixel data survives the round-trip. The same work characterizes the limits of the technique, notably that some steganographic payloads encoded into the pixel values themselves can survive certain round-trips.

These findings directly inform the design of the image re-encoding layer in SecureUploader and, equally importantly, the honest disclosure of its limitations: neutralization by re-encoding reduces

rather than wholly eliminates risk for the narrow class of pixel-level steganographic payloads, and this report states that limit rather than overclaiming.

1.8.6 The Research Gap

The surveyed work leaves a clear gap. Published upload checkers validate format conformance but do not systematically harden against nested-filter and polyglot evasion, nor do they treat neutralization as a measurable outcome. Evasion taxonomies describe attacks but do not provide a hardened, evaluated defense. Machine-learning detectors achieve high accuracy on curated feature sets but require training, operate as opaque classifiers, and do not act on raw bytes at the point of upload. Re-encoding studies establish the sanitization primitive but not an integrated pipeline around it. SecureUploader addresses this gap by combining an integrated, layered, byte-level pipeline with explicit evasion-resistance hardening, a second neutralization metric alongside detection, a transparent per-layer evidence trail, and an empirical evaluation on both a large public corpus and a hand-built adversarial corpus.

1.9 Structure of the Report

The remainder of this report is organized as follows. Chapter Two establishes the theoretical background: the file-upload attack surface, the evasion techniques that defeat naive validation, and the inspection primitives the pipeline composes. Chapter Three documents the methodology and environment, including the platform, the threat model, the evaluation datasets, the host controls, and the metrics used. Chapter Four presents the design and implementation of the nine-layer pipeline. Chapter Five reports the experimental results on both the public and the adversarial corpora. Chapter Six discusses why the design works, the self-bypass findings, the limitations, and the threats to validity. Chapter Seven concludes and outlines future work, and the References follow.

Chapter Two — Theoretical Background

2.1 Introduction

This chapter establishes the technical background needed to understand SecureUploader. It begins with how file formats are structured and how parsers read them, because the disagreements between parsers are the root cause of the most dangerous evasions. It then describes the file-upload attack surface, catalogs the principal classes of attack, and examines the three evasion techniques the pipeline is chiefly designed to defeat — polyglots, nested-filter concealment, and active SVG content. Finally it introduces the inspection building blocks the pipeline composes and the neutralization primitive that distinguishes it, closing with a coverage map and a comparison that motivate the layered design of Chapter Four.

2.2 File Formats and How Parsers Read Them

A file format is a convention for arranging bytes so that a program can recover structured meaning from them. Most formats begin with a magic number — a short, fixed byte sequence that identifies the type, such as FF D8 FF for JPEG, 89 50 4E 47 for PNG, or the literal characters that open a PDF. A parser typically reads this magic number, then follows the format's internal structure of headers, length fields, offsets, and trailing markers to locate the meaningful content.

Crucially, parsers do not all agree on where a file begins and ends. Some read strictly from the leading magic number and stop at a declared length; others scan for a trailing signature and read backwards; still others tolerate leading or trailing junk. This disagreement is not a bug in any one parser but an emergent property of many independent implementations, and it is exactly what an attacker exploits: a single byte sequence can satisfy one parser as a complete, valid image while a second parser, using a different rule, finds and acts on a payload the first ignored.

Two further properties of real formats matter here. First, many formats contain comment or metadata regions whose contents are ignored by the rendering path, giving an attacker a place to hide bytes that survive naive validation. Second, container formats such as PDF allow content to be encoded by a chain of filters, so that the meaningful bytes are not present literally in the file but must be reconstructed by applying each filter in turn. Both properties are leveraged by the evasions described later in this chapter.

2.2.1 JPEG Structure

A JPEG file is a sequence of segments, each introduced by a two-byte marker. It opens with the start-of-image marker (FF D8) and closes with the end-of-image marker (FF D9), and in between carries application and comment segments that hold metadata, followed by the compressed image data. The comment segment is significant for security because its contents are ignored by the rendering path: an attacker can place arbitrary bytes there, and a naive validator that checks only the opening marker will accept the file. Equally important, many JPEG readers stop at the end-of-image marker and ignore anything after it, so bytes appended past FF D9 are invisible to the image while remaining available to a second parser — the basis of the appended-payload and zipper polyglots this project neutralizes by re-encoding.

2.2.2 PNG Structure

A PNG file begins with an eight-byte signature and is then organized as a series of length-prefixed chunks, each carrying a four-character type code and a checksum. Some chunk types are critical to rendering; others, such as textual-metadata chunks, are ancillary and are ignored by the image itself. As with the JPEG comment segment, these ancillary chunks are a natural hiding place for a payload that survives a superficial check, which is why the parasite construction targets them and why rebuilding the image from decoded pixels — which keeps only the critical pixel data — removes them.

2.2.3 PDF Object Model

A PDF is a collection of numbered objects — dictionaries, arrays, and streams — linked by a cross-reference table and organized under a document catalog. Streams may carry a chain of filters that must be applied in order to recover their content, which is the mechanism behind nested-filter concealment. Objects may also declare actions, including automatic actions that fire when the document is opened and actions that launch or fetch external resources, which is the mechanism behind malicious embedded links. This dual structure — filtered streams plus actionable objects — is why the pipeline dedicates a recursive-inflation layer and an embedded-link layer specifically to PDFs, rather than treating them as opaque blobs.

2.2.4 SVG and the XML Model

An SVG file is an XML document whose elements describe vector shapes. Because it is XML, it inherits the full XML feature set, including document-type declarations that can define external

entities, and because it is a web-native image format, browsers execute the script and event handlers it contains when it is rendered. An SVG is therefore simultaneously a picture and a potential program, and unlike a raster image it cannot be reduced to a grid of pixels without losing its meaning. This is why SVG is neutralized not by re-encoding but by parsing the XML and removing the specific dangerous constructs while preserving the drawing, a fundamentally different sanitization strategy from the one used for raster images.

2.3 The File-Upload Attack Surface

An upload endpoint accepts attacker-controlled bytes and typically performs one or more of three dangerous actions with them, each of which turns a property of the file into a capability for the attacker.

- **Storage.** A file written to a path that may later be served or executed becomes dangerous if its bytes are, or can be interpreted as, code. A script stored under a web-executable directory and reachable by URL yields remote code execution.
- **Processing.** A file handed to a parser — an image library, a PDF reader, an XML processor — can trigger memory corruption, entity expansion, or external-resource fetches if the parser is vulnerable or misconfigured, turning a parse into code execution, denial of service, or server-side request forgery.
- **Serving.** A file returned to other users with an active content type executes in their browsers. An SVG or HTML payload served from a trusted origin yields stored cross-site scripting against every viewer.

The essential asymmetry across all three is that the server must decide, from the bytes alone, whether the file is safe, while the attacker is free to construct bytes that satisfy whatever test the server applies. The attack classes below all exploit this asymmetry, and the pipeline in Chapter Four is organized to remove the asymmetry one property at a time.

2.3.1 HTTP Upload Mechanics

A browser typically submits a file using a multipart/form-data request, in which each part carries a small header declaring a field name, a filename, and a content type, followed by the raw bytes of the file. Every one of those declared values is supplied by the client and is therefore attacker-controlled: the filename can be any string, and the content type can be set to any value regardless

of what the bytes actually are. A server that keys its acceptance decision on the declared filename or the declared content type is trusting the attacker to describe the attack honestly, which is why reliable validation must derive the file's type from its bytes rather than from the request metadata.

This distinction between declared and actual type is the seam that several attacks in this chapter exploit. Extension spoofing forges the filename; MIME spoofing forges the content type; and the more advanced polyglot and nested-payload attacks forge the leading bytes themselves so that even a byte-level type check is satisfied. The pipeline therefore treats the request metadata as a hint to be verified, never as a fact to be trusted, and escalates from cheap metadata checks to progressively deeper byte-level inspection.

2.3.2 Storage, Processing, and Serving Paths

What the server does with an accepted file determines which properties become dangerous. If the file is written under a directory from which the web server will execute scripts, then any bytes that can be interpreted as server-side code become a remote-code-execution vector, and the safe practice is to store uploads outside any executable path under server-generated names. If the file is handed to a parser, the parser's own vulnerabilities become reachable by the attacker, so the pipeline prefers to rebuild files rather than merely parse them where possible. And if the file is later served back to users, the content type under which it is served governs whether active content executes, which is why the security-headers middleware described in Chapter Four constrains that serving policy.

The attacker-controlled filename is itself a hazard beyond its extension. A filename containing path-traversal sequences can, on a careless server, cause the file to be written outside its intended directory, and a filename that collides with an existing file can overwrite it. Deriving the stored name from server-side state rather than from the uploaded filename removes this class of problem, and treating the filename purely as an untrusted label — inspected but never used to locate storage — is part of the fail-closed posture the pipeline adopts.

2.4 A Taxonomy of File-Upload Attacks

The principal classes of file-upload attack are summarized in Table 2 and organized as a taxonomy in Figure 1, then examined individually.

A taxonomy of file-upload attacks

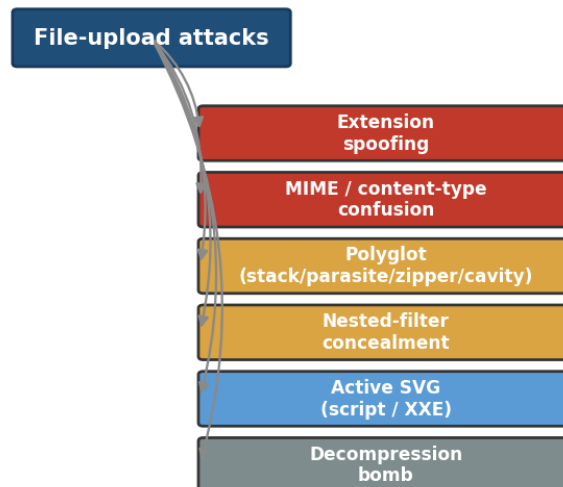


Figure 1: A taxonomy of file-upload attacks

Attack class	Property abused	Typical impact
Extension spoofing	Trust in the filename extension	Web-shell upload, RCE
MIME / content-type spoofing	Trust in the client-supplied MIME type	Bypass of type filters
Polyglot upload	A file valid in two formats at once	Stored XSS, RCE via second parser
Compressed / nested payload	Payload hidden behind stream filters	Signature-scanner evasion
Active SVG content	Script and external entities in SVG	Stored XSS, XXE, SSRF
Decompression bomb	Extreme expansion ratio	Denial of service

Table 2: Principal classes of file-upload attacks and the property each abuses.

2.4.1 Extension Spoofing

The cheapest evasion renames a payload to end in an allowed extension — shell.php becomes shell.jpg, or the double form shell.php.jpg exploits servers that key execution off an interior extension. It defeats any validator that trusts the filename and is neutralized only by inspecting the actual bytes.

2.4.2 MIME / Content-Type Spoofing

The HTTP request that carries an upload includes a client-supplied content-type header. A validator that trusts this header is bypassed by simply setting it to an allowed value, because the

header is attacker-controlled metadata rather than a property of the bytes. Reliable typing requires deriving the type from the content itself.

2.4.3 Web Shells

A web shell is a small server-side script that, once stored under an executable path and requested, gives the attacker a command interface on the host. Web shells are the classic end goal of extension spoofing and image-carried-script polyglots, and they are why storage-path handling and content typing are treated as security-critical rather than cosmetic.

2.4.4 Polyglot Uploads

A polyglot file is simultaneously a valid instance of two or more formats, so that a check for one format passes while a second parser downstream acts on a concealed payload. Polyglots are examined in depth in the next section because they are the central evasion this project targets.

2.4.5 Stored Cross-Site Scripting via Uploads

When an uploaded file is served back with an active content type — most commonly an SVG containing script — the script runs in the browser of every user who views it, from the trusted origin of the hosting site. This turns a passive upload into stored cross-site scripting against the whole user base.

2.4.6 XXE and SSRF via XML-Based Formats

SVG is XML, and XML permits external-entity declarations. A malicious SVG can declare an entity that references a local file or an internal URL; when the document is parsed, the entity is resolved, disclosing file contents (XXE) or causing the server to issue attacker-directed requests (SSRF). The defense is to parse the document and remove the dangerous declarations before it is trusted.

2.4.7 Compression and Decompression Attacks

Two distinct attacks exploit compression. In the first, a payload is hidden behind one or more stream filters so that a single-pass scanner never sees it. In the second, a small file is crafted to expand enormously when decompressed, exhausting memory or disk — a decompression bomb. A robust pipeline must inflate nested content to expose hidden payloads while bounding the work it performs so that it cannot itself be turned into a denial-of-service vector.

2.5 Polyglot Files in Depth

Because format parsers disagree about where a file begins and ends, a single byte sequence can be a valid image to one parser and a valid script or archive to another. Figure 2 illustrates the essential idea: the same bytes on disk are read by an image parser as a legitimate image and by a script engine or archiver as an executable payload.

A single polyglot file (bytes on disk)

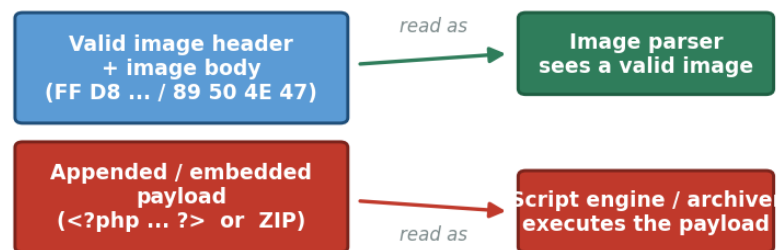


Figure 2: Anatomy of a polyglot file

Polyglots are constructed in a small number of recurring ways. Following the taxonomy popularized by the Mitra tool, the principal constructions are stack, parasite, zipper, and cavity. Each hides the second format in a different structural niche, and each is defeated by a specific pipeline layer, as summarized in Table 3.

Construction	How it works	Defeated by
Stack	Two formats concatenated; parsers start at opposite ends	ImageReEncoding (6)
Parasite	Second format hidden in a comment / metadata chunk	Image re-encoding (drops chunks)
Zipper	Archive appended after a valid image	ImageReEncoding (6) — measured by its trailing-data survey
Cavity	Payload placed in unused / padding bytes	Re-encoding (rebuild from pixels)

Table 3: Polyglot construction taxonomy (after Mitra) and the layer that defeats each class.

The common thread is that in every construction the malicious second format is not part of the image's pixel data. This is the key insight the pipeline exploits: rebuilding the image from its

decoded pixels discards everything that is not a pixel — the appended archive, the parasite chunk, the cavity payload — and so neutralizes the polyglot regardless of which construction produced it.

2.5.1 A Worked Polyglot Example

To make the mechanism concrete, consider a zipper polyglot built by appending a ZIP archive to the end of a valid JPEG. The file begins with the JPEG start-of-image marker and contains a complete, well-formed JPEG that any image viewer renders correctly; after the JPEG's end-of-image marker, the attacker appends the bytes of a ZIP archive. Because a ZIP archive is located by scanning backwards from the end of the file to its central directory, an archive tool reading the same bytes finds and extracts the ZIP, while the image viewer, reading forwards from the start, stops at the end-of-image marker and never notices the appendage. The one file is thus simultaneously a valid image and a valid archive.

This example also shows why no single cheap check suffices. The extension is a legitimate image extension, so whitelisting passes; the leading bytes are a genuine JPEG header, so magic-byte validation passes; the declared type agrees with those bytes, so declared-versus-detected matching passes; and the image renders, so an image-size probe passes. Only one property betrays the file, and only one layer reasons about it: the bytes after the end-of-image marker are not pixel data, so rebuilding the image from its decoded pixels discards the appended archive entirely. The same file that defeats four metadata checks is therefore neutralized by the transformation layer alone — acting on a property the attacker could not forge while keeping the file a valid image.

2.6 Nested-Filter and Compression Concealment

Document formats such as PDF allow a stream to be encoded by a chain of filters — for example several successive FlateDecode passes. A scanner that decodes only the outermost layer, or that pattern-matches the raw stored bytes, never observes the decoded payload. Figure 3 shows the effect: the payload is reached only after the nested filters are inflated in turn.

Recursive inflation of a nested FlateDecode stream

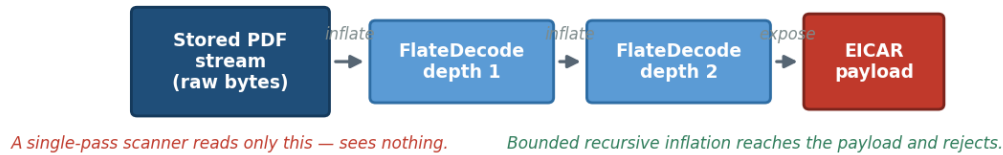


Figure 3: Recursive inflation of a nested FlateDecode stream

Defeating this evasion requires recursively inflating nested filters to a bounded depth before scanning, while guarding against decompression bombs that expand to exhaust memory. The bound on depth and on expanded size is what separates a safe implementation from one that can itself be attacked; Chapter Four describes how the pipeline sets and enforces those bounds.

2.6.1 Decompression Bombs and Expansion Ratios

Recursive inflation introduces a hazard of its own. A decompression bomb is a small input crafted to expand to an enormous size when decompressed — a few kilobytes on disk becoming gigabytes in memory — by exploiting the very high compression ratios achievable on highly redundant data. A pipeline that inflates nested streams without limit can be turned by such an input into a denial-of-service vector against itself, exhausting memory or disk in the course of trying to inspect the file.

The defence bounds two quantities independently, and the two bounds are given deliberately different consequences. Recursion depth is bounded so that an attacker cannot force an unbounded number of inflation passes; on reaching the bound, peeling stops and whatever has been surfaced is scanned. Volume is bounded per stream and per document, and exceeding a volume bound truncates the inflated output and continues the scan rather than refusing the file, because a large legitimate stream is common and refusing on volume alone would manufacture false positives. Refusal is reserved for the expansion ratio: ordinary content compresses by single-digit factors, so a ratio beyond a configured multiple is not a size accident but the signature of a crafted bomb, and is treated as an attack indicator rather than as a limit breach. Distinguishing a size cap from an attack indicator in this way is what converts inflation from a liability into a safe operation, and all

three limits — depth, volume, and expansion ratio — are exposed as configuration so that a deployment can trade inspection depth against resource exposure according to its own risk tolerance.

2.7 Active Content in Vector Formats

SVG is an XML dialect that may contain script elements, event-handler attributes such as `onload`, `foreignObject` nodes embedding arbitrary HTML, and external-entity declarations. When an SVG is served back to users, this active content executes in their browsers, yielding stored cross-site scripting; external entities additionally enable file disclosure and server-side request forgery. Because the danger lies in specific elements and attributes rather than in the file's overall structure, the defense is not to reject SVGs but to parse them and strip the dangerous constructs while preserving the vector drawing — a form of neutralization particular to XML-based formats.

2.8 Inspection Building Blocks

The pipeline is assembled from a small set of inspection primitives, each of which reasons about a different property of the file. None is sufficient alone; their complementary strengths are what make composition worthwhile.

2.8.1 Extension Whitelisting

An allow-list of permitted extensions is the first and cheapest gate. On its own it is trivially bypassed by spoofing, but as the first layer of a pipeline it eliminates the large class of obviously-disallowed types and narrows the set of parsers that later, more expensive layers must consider.

2.8.2 Magic-Byte / Signature Validation

Every well-formed file of a given type begins with a characteristic magic number. Validating that the leading bytes match the claimed extension defeats naive extension spoofing, because a renamed script does not begin with a valid image header. The layer carries signatures not only for the accepted types but for roughly twenty-one forbidden ones — Windows portable executables, ELF binaries, ZIP, RAR, 7z, GIF, TIFF, BMP, CAB, Flash, WebAssembly, PostScript, RTF, OLE compound files, Java class files, gzip archives, script shebangs, and icon containers — so that a refusal identifies what the file actually is rather than merely reporting that it is not what it claimed.

It does not, however, defeat polyglots, whose leading bytes are a genuine image header by construction.

2.8.3 Header–Content Matching

An upload arrives with two independent statements about its type: the Content-Type declared in the request and the type implied by its leading bytes. The first is client-supplied metadata and the second is a property of the content, so a disagreement between them establishes that the declaration was not derived from the file. This check compares the two and refuses a file whose declaration contradicts what the bytes show, which closes the MIME-spoofing path that a magic-byte check alone leaves open — a magic-byte check confirms that the bytes are a valid image, but says nothing about whether the request described them honestly.

The check is bounded in two respects that bear directly on the false-positive rate reported in Chapter Five. First, a browser derives the declared type from the operating system's MIME registry keyed on the filename extension, never from the content; a file that a user has renamed on their own machine therefore produces a genuine disagreement with no hostile intent, and is refused. Misnamed files are common in practice and are an expected source of false positives at this layer. Second, a third statement about type exists — the filename extension itself — and it is compared against neither of the other two: the extension is checked only against an allow-list in Layer 1 and searched for a concealed segment in Layer 4. A file named `photo.png` carrying JPEG bytes and declared `image/jpeg` therefore satisfies every layer, even though extension-versus-content disagreement is precisely the signature of the in-the-wild polyglots catalogued by Koch et al. That disagreement is measured and recorded at this layer but is not made fatal, because making it fatal would change which files the pipeline accepts and that change should follow from corpus evidence on how often benign uploads are simply misnamed, rather than from an assumption adopted at design time.

2.8.4 YARA Signature Scanning

YARA is a rule-based pattern-matching engine widely used to classify files by textual and binary signatures. A curated rule set can flag known-malicious markers — embedded scripts, shell tokens, document actions, or the EICAR test string — anywhere in the content it is given. Signature scanning is precise on known patterns but blind to payloads it has no rule for, which is why it is

one layer among several rather than the whole defense, and why it must be fed decoded rather than raw bytes when payloads are concealed by compression.

2.8.5 Content Re-encoding and Sanitization

The final building block does not merely judge a file; it rewrites it. Decoding an image to its raw pixels and re-encoding it from scratch discards every byte that is not part of the pixel data — appended archives, parasite chunks, and cavity payloads all vanish — while the visible image is preserved. Parsing an SVG and removing script, event handlers, foreignObject nodes, and external entities likewise preserves the drawing while eliminating its active content. This is the mechanism behind neutralization: an ambiguous but potentially useful file is transformed into a provably inert one rather than being rejected.

2.9 Neutralization as a Distinct Outcome

Detection answers a yes/no question: is this file malicious? Neutralization answers a different one: can this file be made safe without discarding its legitimate content? The two are complementary. A file that is clearly malicious should be rejected; a file that is clearly benign should be accepted; but a large middle class of files — an image with appended bytes, an SVG with a stray handler — is neither, and for that class neutralization offers a third outcome that neither rejects a possibly-legitimate upload nor admits a possibly-dangerous one. Treating neutralization as a first-class, measurable outcome rather than a side effect is one of the distinguishing choices of this project, and its limits — chiefly the narrow class of pixel-level steganographic payloads that survive a lossless re-encode — are stated honestly rather than hidden. Those limits are now measured rather than asserted: Section 5.6 reports the effect of an optional bit-clearing step that closes the pixel-domain channel in the lossless case, together with the boundary of that result.

2.10 Mapping Evasion Techniques to Defenses

Anchoring the design in an explicit coverage map makes it possible to state, for each evasion technique, exactly which layer is responsible for defeating it. Table 4 gives that mapping; it is the contract that Chapter Five's adversarial evaluation tests fixture by fixture.

Evasion technique	Defending layer(s)
Extension spoofing	ExtensionWhitelist (1), MagicBytes (2)
MIME / content-type spoofing	MagicBytes (2), HeaderContentMatching (3)
Double / disguised extension	DoubleExtension (4)
Image polyglot (stack / parasite / cavity)	ImageReEncoding (6)
Zipper (appended archive)	ImageReEncoding (6)
Known malicious signature	SignatureScanning (5)
Nested-filter PDF payload	PdfFlateDecode (5b)
Embedded malicious link / action	EmbeddedLink (5c)
Active SVG content (script, handlers, foreignObject)	SvgSanitization (7), by allow-list
XXE / entity expansion	SvgSanitization (7), by DTD prohibition — refusal, not sanitization
Decompression bomb	Bomb guards in (5b) and (6)

Table 4: Mapping of each evasion technique to the pipeline layer(s) that defeat it.

2.11 Comparison of Detection Strategies

No single strategy is sufficient. Cheap metadata checks are easily forged; signature scanning is blind to novel payloads; re-encoding is powerful but applies only to formats that can be safely rebuilt. Their complementary strengths and weaknesses, summarized in Table 5, are precisely what motivates composing them into an ordered pipeline in which each layer covers the blind spot of another.

Strategy	Strength	Weakness
Extension whitelist	Cheap first gate	Trivially spoofed alone
Magic-byte validation	Defeats naive spoofing	Passes polyglots
Header-content matching	Surfaces declared-versus-actual type contradiction	Blind to structure; misnamed benign files are refused
YARA signature scan	Precise on known patterns	Blind to unknown payloads
Recursive inflation	Exposes nested payloads	Cost; bomb risk if unbounded
Re-encoding / sanitization	Neutralizes rather than only detects	Format-specific; rare stego survival

Table 5: Comparison of the core file-inspection strategies used in the pipeline.

2.12 Chapter Summary

File upload is dangerous because the server must judge attacker-controlled bytes, and every cheap check corresponds to a property the attacker can forge. Parser disagreement makes polyglots possible; stream filters make nested concealment possible; XML makes active SVG content possible. The inspection building blocks each cover part of the threat and each have blind spots, which is why the next chapters compose them, in a deliberate order, into a layered pipeline with re-encoding-based neutralization as a distinct outcome, guided by the coverage map of Table 4.

Chapter Three — Methodology and Environment

3.1 Introduction

This chapter documents the methodology and the engineering environment in which SecureUploader was built and evaluated: the research approach, the development platform and libraries, the threat model that bounds what the pipeline is responsible for, the datasets that make up the evaluation corpus, the controlled host environment used for testing with real malicious samples, and the metrics by which results are reported in Chapter Five.

3.2 Research Methodology

The project follows an engineering, design-science methodology: an artifact — the pipeline — is designed to solve a stated problem, built iteratively, and evaluated against explicit criteria. Development proceeded layer by layer, with each layer verified against fixtures that exercise its specific responsibility before the next was added, so that the contribution of each layer could be attributed and, later, ablated. Evaluation was designed up front around two complementary corpora — a large public corpus for aggregate metrics and a hand-built adversarial corpus for evasion resistance — so that neither realistic-but-unengineered files nor engineered-but-unrealistic ones would be relied on alone. The self-bypass mindset, in which each layer is attacked as if by an adversary who knows it exists, was applied throughout and is itself reported as a result in Chapter Six.

3.3 Development Platform and Language

SecureUploader is implemented in C# on .NET 8 using ASP.NET Core. A managed, strongly-typed platform was chosen for three reasons. First, memory safety removes an entire class of vulnerabilities that would otherwise be a concern when parsing untrusted files, so that a malformed input cannot corrupt the pipeline itself. Second, the ASP.NET Core middleware model maps naturally onto an ordered pipeline of inspection layers, each implementing a common interface and each able to short-circuit the request. Third, the ecosystem provides mature, maintained libraries for image processing and pattern matching, avoiding the need to hand-roll parsers for hostile input — which would reintroduce exactly the parsing vulnerabilities the project seeks to avoid.

3.4 Libraries, Tools, and Environment

The implementation relies on a small, deliberately-chosen set of libraries and tools. Image decoding and re-encoding are performed with Magick.NET, upgraded off a known-vulnerable release so that the sanitization primitive is not itself a liability; signature scanning uses a YARA rule set of twenty-four rules including an EICAR rule; and fixture generation for the adversarial corpus uses the open-source Mitra tool. The development and evaluation environment is summarized in Table 6.

Component	Choice
Language / runtime	C# on .NET 8
Web framework	ASP.NET Core (middleware pipeline)
Image processing	Magick.NET (patched release)
Signature engine	YARA (24 rules, incl. EICAR)
Fixture generation	Mitra (Ange Albertini)
Frontend	React instrument-panel UI
Host OS (testing)	Windows, hardened lab configuration

Table 6: Development platform, language, and key libraries.

3.5 Threat Model

A defense is only meaningful relative to an explicit threat model. In this project the attacker controls every byte of the uploaded file and every character of its filename, and acts solely through the public upload endpoint, as depicted in Figure 4. The attacker's goals include remote code execution, stored cross-site scripting, XXE and SSRF, denial of service, and the hosting of malware on the trusted origin. Out of scope are attacks that do not pass through the endpoint — network interception, compromise of the host operating system, or an insider with direct server access — and the pipeline and its host are assumed trusted, with the one documented exception that host antivirus can interfere with measurement, which is addressed in the host controls.

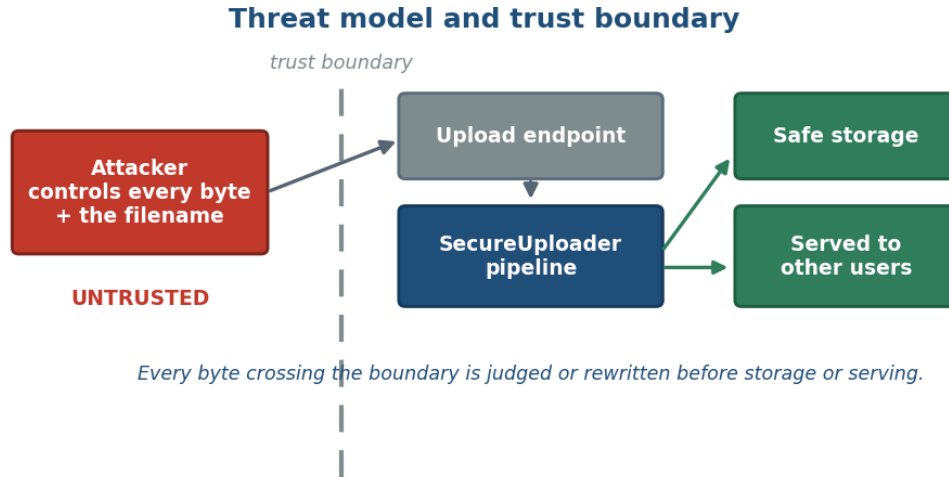


Figure 4: Threat model and trust boundary

Aspect	Assumption
Attacker control	Full control of file bytes and filename
Attacker goals	RCE, stored XSS, XXE / SSRF, DoS, malware hosting
Delivery channel	The public upload endpoint only
Trusted components	The pipeline and the analysis host
Out of scope	Network interception, host OS compromise, insider access

Table 7: Threat-model assumptions: attacker capabilities, goals, and what is out of scope.

3.6 Evaluation Datasets and Corpus Construction

Evaluation uses two complementary corpora. A large public corpus supplies aggregate detection statistics on realistic malicious and benign documents, and a smaller hand-built adversarial corpus stress-tests the pipeline against inputs constructed specifically to evade it. The datasets and their roles are summarized in Table 8.

Dataset	Role in evaluation
CIC-Evasive-PDFMal2022	10,012-file aggregate corpus (5,549 malicious / 4,463 clean)
Koch et al. polyglot catalog	Real polyglot samples (by hash) for the adversarial corpus
MalwareBazaar (abuse.ch)	Sole download route for real malicious samples
Kaggle natural-images	Benign image set for false-positive testing
Inert fixtures (Mitra)	Crafted safe polyglots and evasion fixtures

Table 8: Evaluation datasets and their roles in the corpus.

3.6.1 The Public Corpus (CIC-Evasive-PDFMal2022)

The primary quantitative evaluation uses the full CIC-Evasive-PDFMal2022 corpus of 10,012 files, comprising 5,549 malicious and 4,463 clean PDF documents. The dataset is designed to include evasive malicious PDFs — documents built to slip past detectors — which makes it a demanding test of detection rather than a collection of trivially-flagged samples, and therefore a fair basis for the aggregate metrics reported in Chapter Five. It is worth stating a property of the benign half up front: the clean documents in this corpus were not sampled at random but selected to resemble the malicious class, so that byte-level and signature indicators are, by construction, weak discriminators on the clean set. This makes the corpus a deliberately adversarial basis for the false-positive analysis, and the false-positive results in Chapter Five should be read with that construction in mind rather than as an estimate over a random real-world upload population.

3.6.2 Real Polyglots and the Download Route

Real polyglot samples are drawn by hash from the Koch et al. catalog and retrieved exclusively through MalwareBazaar using a free authentication key. Restricting downloads to a single sanctioned route keeps the handling of live malware auditable. Some older hashes are unavailable through that route; these are logged as a data-availability limitation rather than silently dropped, so that the size of the real-sample portion of the corpus is reported honestly.

3.6.3 Benign Corpus and Inert Fixtures

Legitimate images from the Kaggle natural-images set provide the benign population for false-positive measurement, giving a realistic distribution of the images a real deployment would accept. A set of inert fixtures generated with Mitra provides crafted polyglots and evasion cases whose behavior is fully known in advance, which is what makes the adversarial evaluation interpretable: for each fixture the expected verdict is known, so a wrong verdict is unambiguous.

Corpus construction followed a few disciplines that keep the evaluation honest. Real samples are identified by their SHA-256 hash and retrieved by that hash, so there is no ambiguity about which sample a result refers to and no risk of silently substituting a different file. Samples are deduplicated by hash so that an identical file cannot be counted twice and inflate a rate. And for every category the count actually obtained is recorded rather than the count intended, so that when some hashes prove unavailable the reported corpus size reflects what was truly tested. These

practices matter because a detection rate is only as trustworthy as the inventory it is computed over.

The two corpora play deliberately different roles and are never blended. The public corpus provides a population-level view: a balanced set of realistic malicious and clean documents over which aggregate metrics are meaningful. The adversarial corpus provides a capability-level view: a set of engineered fixtures over which the question is not what fraction is caught in a realistic population but whether each specific evasion is defeated. Reporting them separately, with the metrics appropriate to each, avoids the error of reading an evasion-resistance result as a population estimate or vice versa.

3.7 Containment and the Handling of Real Malware

Because part of the corpus consists of real malicious samples, containment was treated as a first-class methodological concern rather than an afterthought. Two dangerous real polyglots — a JPEG-plus-ZIP and a PNG-plus-RAR sample associated with known malware families — are deliberately not downloaded at all; an inert equivalent that reproduces the structural evasion without the live payload is built instead, and this containment decision is documented so that the choice is transparent. Samples that are downloaded are handled only under supervisor approval and only within the hardened host configuration described next.

3.8 Accepted File Types and Threat Scope

SecureUploader accepts a deliberately narrow set of file types. Office documents and archive formats are excluded because their macro and container semantics broaden the attack surface beyond what the pipeline is designed to neutralize. The accepted types and their in-scope threats are listed in Table 9.

Type	Extensions	In-scope threats
Raster image	.jpg .jpeg .png .webp	Polyglots, appended payloads, parasite chunks
Vector image	.svg	Script, event handlers, foreignObject, XXE
Document	.pdf	Nested-filter payloads, embedded links / actions

Table 9: Accepted file types and their in-scope threat classes.

GIF was removed from an earlier accepted set and WebP added; no Office or archive types are accepted. Because the accepted types are almost entirely non-executable on the host operating system, this narrow scope is also what makes controlled testing on the host machine defensible: a sample that cannot execute on the host cannot compromise it merely by being present.

3.9 Testing Environment and Host Controls

A full virtual machine proved unusably slow on the available hardware, so testing is performed on the host with layered controls that together substitute for isolation. Samples are neutralized on disk by appending an inert suffix that the harness strips before submission, so a stored sample is never a live file of its claimed type; the suffix must be removed before the file enters the pipeline because the filename itself is an input to Layers 1, 3, and 4, so scanning the suffixed name would have tested a different program; folder previews, thumbnails, and search indexing are disabled for the lab directory so that no background process opens a sample; automatic sample submission is turned off; and real-time protection remains enabled everywhere except for a single narrowly-scoped exclusion on the scanner's temporary directory.

That one exclusion resolved a subtle measurement error worth stating in full, because it is also reported as a self-bypass finding in Chapter Six. The recursive PDF-inflation layer writes decompressed streams to a temporary file before scanning them; the host antivirus quarantined EICAR payloads in that temporary file before YARA could read them, causing genuine detections to be recorded as failures — the pipeline was working, but the environment was hiding its successes. Routing all scanner temporary files to a single excluded directory, with real-time protection otherwise on everywhere, removed the interference without weakening host protection.

3.9.1 Determinism and Reproducibility

A property that runs through both the design and the evaluation is determinism: the pipeline uses no randomness, no time-dependent behavior, and no learned model, so the same input always produces the same verdict and, for a neutralized file, the same rewritten bytes. This is what makes the evaluation reproducible — a reported verdict can be regenerated exactly — and it is what allows the pipeline to be used inside an automated build without introducing nondeterministic failures.

Determinism also disciplines the neutralization layers. Because re-encoding an image or sanitizing an SVG must yield identical output for identical input, the transformation cannot depend on any

ambient state, which in turn makes the transformed file safe to cache and safe to compare byte-for-byte across runs. The 28,554-to-14,863-byte reduction reported for the appended-payload fixture in Chapter Five is not an average over runs but a fixed result that the same input reproduces every time.

The claim is checkable rather than asserted. The re-encoded output of the transformation layer is hashed with SHA-256 and that digest is recorded in the evidence trail alongside the before-and-after sizes, so re-running the same input reproduces the same digest and a reviewer can verify determinism from the published results rather than taking it on trust.

3.10 Evaluation Metrics

Results in Chapter Five are reported with the standard classification metrics, defined in Table 10 in terms of true and false positives and negatives (TP, FP, TN, FN). A positive is a file the pipeline flags as malicious; recall measures how many malicious files are caught, precision how many flagged files are truly malicious, and the false-positive rate how many clean files are wrongly flagged. Reporting all of them together, rather than a single headline number, is what makes the trade-off between coverage and false positives visible.

Metric	Definition	Formula
Accuracy	Fraction of all files classified correctly	$(TP+TN) / (TP+TN+FP+FN)$
Precision	Fraction of flagged files that are truly malicious	$TP / (TP+FP)$
Recall	Fraction of malicious files that are caught	$TP / (TP+FN)$
Specificity	Fraction of clean files correctly accepted	$TN / (TN+FP)$
F1-score	Harmonic mean of precision and recall	$2 \cdot P \cdot R / (P+R)$
False-positive rate	Fraction of clean files wrongly flagged	$FP / (FP+TN)$

Table 10: Definitions of the evaluation metrics used in Chapter Five.

3.11 Chapter Summary

SecureUploader is built on a memory-safe, middleware-oriented platform with a small set of mature libraries, under an explicit threat model in which the attacker controls the uploaded bytes and acts only through the endpoint. It is evaluated on a balanced public corpus and a hand-built adversarial corpus, restricted to a narrow set of near-non-executable file types, tested under a hardened host configuration whose one measurement pitfall was identified and corrected, and

reported with the standard classification metrics. Chapter Four turns to the design of the pipeline itself.

Chapter Four — System Design and Implementation

4.1 Introduction

This chapter presents the design and implementation of the SecureUploader pipeline: its guiding principles, the abstraction and orchestration that let independent layers compose, the nine layers themselves grouped by function and examined individually, the caching layer that makes repeated scanning cheap, the security-headers middleware and configuration that govern its behavior, the verdict and evidence-trail model, and the frontend through which its decisions are inspected.

4.2 Design Principles

Five principles govern the design.

- **Defense in depth.** No file is trusted until it has passed every applicable layer, and each layer reasons about a different property, so that defeating the pipeline requires defeating all applicable layers at once.
- **Fail closed.** Ambiguity resolves toward safety: a file that cannot be shown safe is rejected or neutralized rather than accepted, and an error in a layer stops the pipeline rather than letting the file through.
- **Ordering by cost.** Layers run from cheapest and most general to most expensive and most specific, so most rejected files are eliminated early and the costly layers run only on inputs that survive the earlier gates.
- **Determinism.** The same input always yields the same verdict and, for neutralization, the same rewritten output, which is a prerequisite for reproducibility and for use inside automated pipelines.
- **Transparency.** Every verdict carries a per-layer evidence trail, so a decision is auditable rather than an opaque score.

4.3 High-Level Architecture

The pipeline comprises nine layers in three functional groups: structural layers that check metadata and format coherence, detection layers that scan decoded content for malicious patterns, and transformation layers that rewrite a file to neutralize a payload. The overall data flow, from an untrusted upload to one of three verdicts, is shown in Figure 5, and the layers are enumerated in Table 11.

SecureUploader Pipeline Architecture

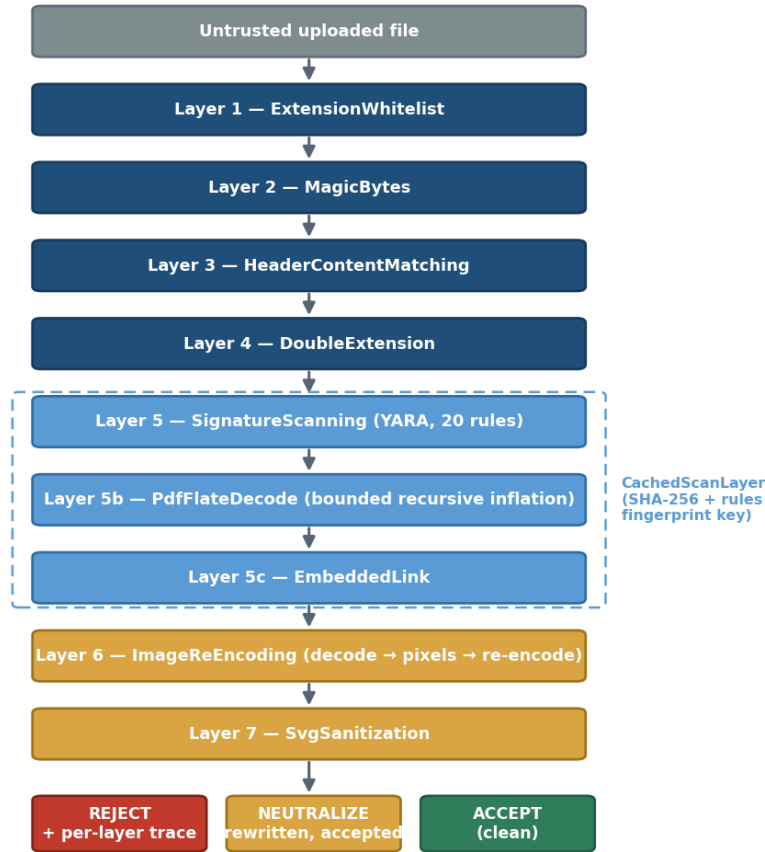


Figure 5: SecureUploader pipeline architecture

#	Layer	Group	Responsibility
1	ExtensionWhitelist	Structural	Reject disallowed extensions
2	MagicBytes	Structural	Match leading bytes to the claimed type
3	HeaderContentMatching	Structural	Match the declared Content-Type to the detected type
4	DoubleExtension	Structural	Detect double / disguised extensions
5	SignatureScanning	Detection	YARA scan (24 rules, incl. EICAR)
5b	PdfFlateDecode	Detection	Bounded recursive stream inflation, then scan
5c	EmbeddedLink	Detection	Inspect embedded links / actions
6	ImageReEncoding	Transformation	Decode to pixels and re-encode (neutralize)
7	SvgSanitization	Transformation	Strip script, handlers, foreignObject, XXE

Table 11: The nine inspection layers, their category, and their responsibility.

4.4 The IScanLayer Abstraction and Orchestration

Every inspection layer implements a single interface, `IScanLayer`, exposing one operation that takes a scan context — the file stream, the filename, the declared content type, the type detected from the bytes, and the accumulating trace — and returns a per-layer result. This uniformity is what lets the layers compose: the orchestrator holds an ordered list of layers and runs each in turn, as shown in Figure 6. A layer may accept the file and pass it on, refuse it and short-circuit the remainder of the pipeline, or — for the transformation layers — return a sanitized stream that every later layer then reads instead of the original. Because a refusal short-circuits, the expensive detection and transformation layers never run on a file already eliminated by a cheap structural check.

Two interfaces are involved and should not be conflated. Layers implement `IScanLayer` and are composed by the pipeline. The six validators compared in Chapter Five — the four DVWA reference levels, the baseline, and the layered filter presented as a single unit — implement a separate interface, `IFileScanner`, whose operation takes a whole uploaded file and returns a scanner-level verdict. That separation is what allows the comparison harness to hand identical bytes to the layered pipeline and to a flat reference validator under identical conditions, while keeping the internal composition of the pipeline invisible to the harness. The comparators are registered by their concrete types and never under the shared interface, so the single registration that selects the production scanner cannot become ambiguous and silently change which validator serves real uploads.

Pipeline orchestration via the `IFileScanner` interface

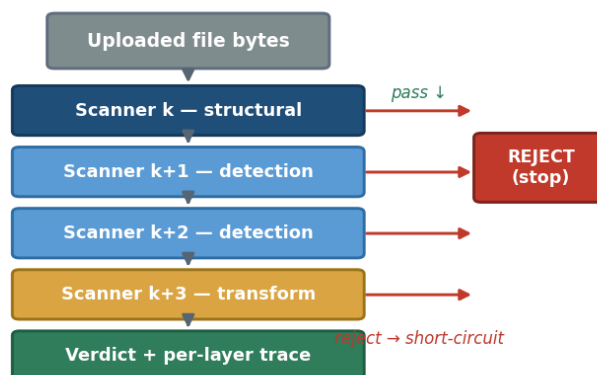


Figure 6: Pipeline orchestration via `IScanLayer`

A second abstraction underpins evaluation. The reference validators used in Chapter Five — the DVWA-style scanners and the basic baseline — implement `IFileScanner`, the scanner-level interface described above, so the comparison orchestrator can run identical bytes through every validator in turn and record each verdict and trace under identical conditions.

```
// Every inspection layer implements this one interface, which is what lets the
// orchestrator treat all nine uniformly.
public interface IScanLayer
{
    string      Name { get; }
    ScanResult Scan(FileScanContext context);
}

public enum ScanDecision
{
    Accepted,    // passed this layer
    Rejected,    // failed this layer - the pipeline stops
    Sanitized    // rewritten by this layer - the pipeline continues on the new stream
}

// One observation behind a decision. A reason states the CONCLUSION; evidence
// states the OBSERVATIONS it rests on, so a verdict can be checked, not trusted.
public sealed record ScanEvidence(
    string Kind,                // groupable class: "yara-rule",
                                // "reencode-trailing", ...
    string Label,               // what was observed: rule name, field, element
    string Detail,              // the observed value, in readable form
    long? Offset = null,        // byte position, where the observation has one
    string? Severity = null,     // sourced from the detecting RULE, never invented here
    string? Reference = null);  // CVE / CWE / ATT&CK id / paper

public sealed class ScanResult
{
    public ScanDecision Decision { get; }
    public string      LayerName { get; }
    public string?     RejectionReason { get; } // logged server-side,
                                                // never returned
    public Stream?     SanitizedStream { get; } // set only when Sanitized
    public IReadOnlyList<ScanEvidence> Evidence { get; }
    public bool        FromCache { get; private set; }

    // A private constructor forces the factory methods, so an invalid
    // combination (Rejected without a reason) cannot be constructed.
    public static ScanResult Accept(string layer,
                                    IReadOnlyList<ScanEvidence>? e = null);
    public static ScanResult Reject(string layer, string reason,
                                    IReadOnlyList<ScanEvidence>? e = null);
    public static ScanResult Sanitize(string layer, Stream cleaned,
                                      IReadOnlyList<ScanEvidence>? e = null);
}
```

Listing 1: The `IScanLayer` interface, the decision enumeration, and the evidence-carrying result type.

```

public ScanResult ScanFile(FileScanContext context)
{
    foreach (var layer in _layers)           // ordered: cheap -> expensive
    {
        if (context.FileStream.CanSeek)
            context.FileStream.Position = 0;    // every layer reads from the start

        // Timed on EVERY path, rejection included: the cost of an early reject is
        // what justifies the cheap-checks-first ordering, so it is measured.
        var sw = Stopwatch.StartNew();
        var result = layer.Scan(context);
        sw.Stop();

        // The trace is written HERE, not by the layer. No layer knows it exists,
        // so none can forget to populate it or report it inconsistently.
        context.Traces.Add(new LayerTrace
        {
            Layer      = layer.Name,
            Decision    = result.Decision.ToString(),
            Reason      = result.RejectionReason,
            ElapsedMs   = sw.Elapsed.TotalMilliseconds,
            FromCache   = result.FromCache,
            Evidence    = result.Evidence
        });

        switch (result.Decision)
        {
            case ScanDecision.Rejected:
                return result;                               // short-circuit
            case ScanDecision.Sanitized:
                // continue on the rewrite
                context.FileStream = result.SanitizedStream!;
                break;
            case ScanDecision.Accepted:
                break;
        }
    }
    return ScanResult.Accept("Pipeline");
}

```

Listing 2: The orchestration loop: ordered execution, timing, tracing, and short-circuit on refusal.

The interface, shown in Listing 1, keeps every layer to the same shape: it names itself, receives the scan context, and returns a decision together with the observations behind it and, for a transformation layer, the rewritten stream. The three-valued decision is what allows a layer to accept, refuse, or rewrite without any layer needing to know about the others, and the factory methods make an inconsistent result — a refusal with no reason — impossible to construct.

The orchestrator, shown in Listing 2, is deliberately small: it iterates the ordered layers, rewinds the stream before each one, times every call, writes the trace entry itself rather than trusting the layer to do so, short-circuits on the first refusal, and substitutes any sanitized stream so that a rewrite is visible to everything downstream.

4.4.1 Extending the Pipeline with a New Layer

The uniform interface makes the pipeline straightforward to extend. Adding a new inspection is a matter of implementing `IScanLayer` and registering the implementation at the appropriate position in the ordered list; no existing layer needs to change, because each layer is oblivious to the others and communicates only through the shared result type and trace. This is a direct benefit of the abstraction: a new defense for a new format, or a stricter check for an existing one, can be developed and tested in isolation and then slotted into the sequence at a point consistent with its cost.

The same independence is what makes the ablation study planned in Chapter Seven possible. Because a layer can be removed from the ordered list without disturbing the rest, the marginal contribution of any single layer to detection and to neutralization can be measured by running the corpus with and without it. The architecture thus serves not only the running system but also the empirical study of the system, which is a deliberate design goal rather than an accident.

4.5 Structural Layers (1–4)

The first four layers reason about metadata and format structure without decoding content, and are cheap enough to run on every upload.

4.5.1 ExtensionWhitelist (Layer 1)

The first layer rejects any file whose extension is outside the accepted set. It is trivially bypassable in isolation, but as the first gate it removes the large class of obviously-disallowed types and fixes which parsers the later layers must consider, so that subsequent layers can make format-specific assumptions safely.

4.5.2 MagicBytes (Layer 2)

The second layer reads the leading bytes and verifies that they match the claimed type — a JPEG must begin with the JPEG magic number, a PNG with the PNG signature, and so on. This defeats naive extension spoofing, because a renamed script or a header-less payload does not begin with a valid image header, and it is the layer that rejects the header-less fixture in Chapter Five. The signature table is not limited to the accepted types: it also carries the signatures of roughly twenty-one forbidden ones, so that a refusal identifies the real type of the content — "this is a Windows executable submitted as a .png" — rather than reporting only that no accepted signature matched. The type it detects is written into the scan context and is the input the next layer compares against.

```

// Layer 2 - the leading bytes decide the type. The table carries signatures for
// the ACCEPTED types and for ~21 FORBIDDEN ones, so a refusal can name what the
// file actually is instead of only reporting that it is not what it claimed.
private sealed record FileSignature(string Type, int Offset, byte[] Magic);

private static readonly FileSignature[] Accepted =
{
    new("jpeg", 0, new byte[] { 0xFF, 0xD8, 0xFF }),
    new("png", 0, new byte[] { 0x89, 0x50, 0x4E, 0x47 }),
    new("webp", 0, "RIFF"u8.ToArray()), // plus "WEBP" at offset 8
    new("pdf", 0, "%PDF-"u8.ToArray()),
    // svg is XML: located by matching the opening marker in the leading bytes
};

private static readonly FileSignature[] Forbidden =
{
    new("Windows PE", 0, "MZ"u8.ToArray()),
    new("ELF binary", 0, new byte[] { 0x7F, 0x45, 0x4C, 0x46 }),
    new("ZIP archive", 0, new byte[] { 0x50, 0x4B, 0x03, 0x04 }),
    new("RAR archive", 0, "Rar!"u8.ToArray()),
    new("OLE compound file", 0, new byte[] { 0xD0, 0xCF, 0x11, 0xE0 }),
    // ... 7z, GIF, TIFF, BMP, CAB, Flash, WASM, PostScript, RTF, Java class,
    //      gzip, script shebang, ICO
};

// A forbidden match is reported BY NAME. Only if none matches is the absence of
// an accepted signature treated as an unrecognised type.
if (FirstMatch(head, Forbidden) is { } bad)
    return ScanResult.Reject(Name,
        $"{ReasonForbidden}: content is {bad.Type}.", evidence);

if (FirstMatch(head, Accepted) is not { } good)
    return ScanResult.Reject(Name,
        $"{ReasonUnknown}: no accepted signature.", evidence);

context.DetectedFileType = good.Type; // the input Layer 3 compares against
return ScanResult.Accept(Name, evidence);

```

Listing 3: The signature table, including forbidden types, and the type written into the scan context (Layer 2).

4.5.3 HeaderContentMatching (Layer 3)

The third layer verifies that the Content-Type declared by the client is consistent with the real type written into the scan context by Layer 2. The layer therefore has a hard ordering dependency on Layer 2 and, in a fail-closed posture, refuses the file when the detected type is absent — a missing detection indicates that the pipeline was misconfigured rather than that the file is safe. Three outcomes are distinguished in the evidence trail: an exact agreement, an agreement through a documented alias (for example image/jpg for image/jpeg), and a contradiction. Only the third is fatal.

The layer additionally records, without acting on it, whether the filename extension agrees with the detected type. That observation is reported so it can be tallied over the corpus, and the decision not to reject on it is stated in Section 2.8.3.

```
// Layer 3 - declared Content-Type vs the type Layer 2 detected from the bytes.
// Fail-closed: an absent detection is a pipeline fault, not an acceptance.
if (context.DetectedFileType is null)
    return ScanResult.Reject(Name, $"{ReasonPipeline}: Layer 2 did not run.");

var declared = Normalize(context.DeclaredContentType);
var expected = ContentTypesFor(context.DetectedFileType); // canonical + aliases

if (!expected.Contains(declared))
    return ScanResult.Reject(Name,
        $"{ReasonMismatch}: declared '{declared}' contradicts detected " +
        $"{context.DetectedFileType}'.", evidence);

// Third signal: extension vs content. MEASURED AND REPORTED, never fatal -
// see Section 2.8.3 for why.
evidence.Add(DescribeExtensionAgreement(context.FileName, context.DetectedFileType));
return ScanResult.Accept(Name, evidence);
```

Listing 4: Declared-type against detected-type matching, with the extension signal recorded but not fatal (Layer 3).

4.5.4 DoubleExtension (Layer 4)

The fourth layer detects disguised names such as a script extension hidden before an allowed one, closing the double-extension bypass against servers that key execution off an interior extension. Together the four structural layers eliminate the cheapest and most common evasions and narrow the input reaching the detection layers to well-formed files of an accepted type.

4.5.5 Error Handling and Fail-Closed Behavior

Parsing untrusted input is inherently error-prone, and how the pipeline responds to an error is a security decision in its own right. The pipeline fails closed: if a layer throws while inspecting a file — a malformed structure that crashes a parser, an inflation that trips a resource limit, an image that cannot be decoded — the file is rejected rather than accepted, and the error is recorded in the trace with the layer that produced it. An attacker therefore cannot benefit by crafting an input that breaks a layer, because a broken layer yields a rejection, not a pass. One deliberate asymmetry in this posture is stated where it arises: exhausting the bounded inflation depth of Layer 5b is not treated as an error and does not refuse the file, for the reasons given in Section 4.6.2 and recorded as a limitation in Section 6.8.

This posture extends to resource limits. The depth and expansion bounds on inflation, the pixel bound on re-encoding, and the size limits on upload are all enforced as hard limits whose violation is a rejection, so that no single file can consume unbounded work. The combination of fail-closed error handling and hard resource limits is what allows the pipeline to inspect hostile input safely: a file can be malformed, oversized, or actively adversarial, and the worst outcome is a recorded rejection rather than a crash or an unbounded computation.

4.6 Detection Layers (5, 5b, 5c)

4.6.1 YARA Signature Scanning (Layer 5)

Layer 5 scans the file with a curated YARA rule set of twenty-four rules, including a rule for the EICAR test string. Signatures match known-malicious markers — embedded scripts, shell tokens, document actions, and known payload fragments — anywhere in the content it is given. Every rule declares its own severity in its metadata, and that declared severity — not an interpretation made by the layer — is what the evidence trail records and what the acceptance policy discussed in Section 5.3.3 is defined over. The rule categories, their counts, and the severities they declare are summarized in Table 12. Because signature scanning is only as good as the content it is shown, its effectiveness against concealed payloads depends on the inflation performed by the next layer.

Table 12: Categories of YARA rules in the signature-scanning layer, with their counts and declared severities.

Category	Rules	Declared severity	Targeted indicators
Test markers	1	test	The standard antivirus test string; excluded from detection figures
PHP web shells	4	critical	Input reaching an execution sink; obfuscated execution; known shell signatures; PHP carried inside an image container
Other web shells	3	critical	ASP, ASPX, and JSP execution primitives
PDF actions and structure	7	high, medium	Auto-executing JavaScript; launch actions; embedded files; obfuscated name encoding; JavaScript via a name tree; page-covering link; encrypted document
Office and DDE	3	high	Auto-executing macros; suspicious macro calls; dynamic data exchange on open
Active markup	3	high, medium	Script elements and event-handler attributes; active content in a vector image; external-entity declarations
Embedded executables	2	critical	Windows portable-executable and ELF headers inside an accepted container
Embedded archives	1	critical	Archive signatures found inside an image container
Total	24		

4.6.2 PDF FlateDecode Recursive Inflation (Layer 5b)

Layer 5b addresses nested-filter concealment. A PDF stream may be encoded by a chain of FlateDecode filters; a scanner that decodes only the outermost layer never observes the payload. This layer locates every stream in the document, inflates what it can, and hands the decompressed content to the signature engine — the same engine invocation used by Layer 5, shared deliberately

so that any difference between the two layers' results is attributable to what was scanned rather than to a divergence in how the scanner was called.

The layer is deliberately not a specification-compliant PDF parser. A security filter needs to surface concealed content, not to render the document, so inflation is attempted on every stream blob on a best-effort basis, with several strategies at the first pass and strict inflation at deeper ones. Two properties follow from that choice: no heavyweight PDF library is exposed to hostile input, which would reintroduce the parsing vulnerabilities the platform was chosen to avoid; and the layer errs toward decompressing more rather than less.

Two independent bounds guard the inflation, and they behave differently by design. Depth is bounded by `MaxDecodeDepth`, set to four, so an attacker cannot force an unbounded number of passes; when the bound is reached, peeling stops and whatever has been surfaced is scanned. Volume is bounded twice, by `MaxBytesPerStream` and `MaxTotalBytes`, and these bounds truncate and continue scanning rather than refusing the file: a large legitimate stream is entirely plausible, so refusing on size alone would manufacture false positives. Refusal is reserved for the expansion ratio: `MaxExpansionRatio`, set to 1500, refuses the file outright, because ordinary content compresses by single-digit factors and an extreme ratio is not a size accident but the signature of a crafted bomb. Separating a size cap from an attack indicator in this way is what allows the layer to inspect deeply without becoming a denial-of-service surface itself.

Because signature offsets index a concatenated in-memory buffer rather than the upload, the span of every decompressed stream is recorded and each match offset is translated back into the stream it came from and that stream's position in the original file. This is what turns a claim of a match at nesting depth two into a checkable statement: the payload lay inside the stream beginning at a named offset, required a stated number of inflation passes to reach, and expanded by a measured factor — and was therefore invisible to the raw scan in Layer 5, which is the ablation argument for this layer's existence.

Two fixtures demonstrate the effect and are analyzed in Chapter Five: an EICAR payload behind a single `FlateDecode` is caught after one level of inflation, and the same payload behind a double `FlateDecode` is caught after two — the latter having passed plain signature scanning, which is precisely the ablation argument for why this layer exists as a separate stage rather than being folded into Layer 5.

```

// One stream, peeled repeatedly. The depth bound STOPS the peeling; the ratio
// bound REFUSES the file. See the text for why the two differ.
private InflateOutcome InflateRepeatedly(byte[] streamBytes)
{
    var outcome = new InflateOutcome();
    var current = streamBytes;
    int depth = 0;

    while (depth < _limits.MaxDecodeDepth)
    {
        // Depth 0 tries every strategy to maximise what is surfaced; deeper
        // passes use strict inflation only.
        var next = TryInflateOnce(current, strictOnly: depth > 0);
        if (next is null) break; // nothing further to peel

        depth++;
        current = next;

        long ratio = streamBytes.Length == 0 ? 0 : current.Length / streamBytes.Length;
        if (ratio > _limits.MaxExpansionRatio) // decompression-bomb INDICATOR
        {
            outcome.BombDetected = true;
            outcome.Depth = depth;
            outcome.Reason =
                $"PDF stream expanded {ratio}x (limit {_limits.MaxExpansionRatio}x) " +
                $"at nesting depth {depth} - decompression-bomb indicator.";
            return outcome;
        }
    }

    outcome.Data = depth > 0 ? current : Array.Empty<byte>();
    outcome.Depth = depth;
    return outcome; // byte caps are enforced inside TryInflateOnce, which
                  // TRUNCATES rather than refusing (see the text)
}

```

Listing 5: Bounded repeated inflation: the depth bound stops peeling, the expansion ratio refuses the file (Layer 5b).

The inflation routine, shown in Listing 5, is where the asymmetry between the bounds is visible. The loop condition enforces the depth bound, so exhausting it simply ends the peeling and the content surfaced so far is scanned; the byte caps are applied inside the single-pass helper, which truncates its output and lets the scan continue; and only the expansion ratio produces a refusal, because only that quantity distinguishes a crafted bomb from a large legitimate stream.

4.6.3 Embedded-Link Inspection (Layer 5c)

Layer 5c inspects the references a document carries. Rather than walking a parsed object model, it extracts text from the raw content and matches seven dangerous URI schemes — javascript, vbscript, livescript, mocha, file, php, and jar — together with three executable inline-data prefixes. The schemes are matched with an anchored expression that only fires where a URI can begin; an earlier substring search matched file: inside the ordinary word Profile:, which is one of the self-bypass findings recorded in Chapter Six.

Two properties of the layer are policy rather than capability, and both are disclosed. In the evaluated configuration the external-link block is off, so ordinary http and https references are recorded in the evidence and permitted rather than refused, because refusing every external reference carries a benign-collision rate a deployment should choose rather than inherit. And the layer has two blind spots that are recorded alongside its result so that a clean finding is never mistaken for a complete one: references inside compressed document streams are not extracted, and references are not character-decoded before matching, so a scheme expressed through character references or percent-encoding in a vector document is not matched.

```
// Layer 5c - dangerous URI schemes, matched only where a URI can BEGIN.
// A bare substring search made "file:" match inside "Profile:" - a self-bypass
// finding recorded in Table 20.
private static readonly string[] DangerousSchemes =
    { "javascript", "vbscript", "livescript", "mocha", "file", "php", "jar" };

private static readonly Regex SchemeRegex = new(
    @"(?<![A-Za-z0-9+.\-])(\" + string.Join('|', DangerousSchemes) + @)":",
    RegexOptions.IgnoreCase | RegexOptions.Compiled);

// Executable inline data is refused by prefix, not by scheme.
private static readonly string[] ExecutableDataPrefixes =
    { "data:text/html", "data:image/svg+xml", "data:application/javascript" };

foreach (Match m in SchemeRegex.Matches(text))
    evidence.Add(new ScanEvidence("embedded-link", m.Groups[1].Value,
        Preview(text, m.Index, m.Index, "high")));

// Ordinary http(s) references are RECORDED, not refused: blockExternalLinks is
// false in the evaluated configuration (see Section 6.8).
if (evidence.Any(e => e.Severity == "high"))
    return ScanResult.Reject(Name,
        $"{ReasonScheme}: dangerous scheme in a reference.", evidence);

// Disclosed blind spots: references inside compressed streams are not extracted,
// and character references / percent-encoding are not decoded before matching.
return ScanResult.Accept(Name, evidence);
```

Listing 6: Anchored dangerous-scheme matching over extracted text, with external references recorded rather than refused (Layer 5c).

4.7 Transformation Layers (6, 7)

4.7.1 Image Re-encoding (Layer 6)

Layer 6 replaces an earlier metadata-inspection layer with an active defence. An accepted raster image is decoded to its pixels and re-encoded from scratch, so that only pixel data survives the round-trip: appended archives, parasite metadata chunks, and cavity payloads are destroyed structurally rather than detected. Because the malicious second format in every polyglot

construction lies outside the pixel data, rebuilding from pixels removes it irrespective of which construction produced it.

Decoding untrusted input is itself a risk, and four controls bound it. First, the image dimensions are read from the header alone, using a header-only inspection, and are refused against separate width, height, and total-pixel limits before any pixel is decoded, so a decompression bomb is rejected without the decode cost ever being paid. Second, library-wide memory, width, and height ceilings are configured at startup as a backstop behind that check. Third, the decode format is pinned from the type detected by Layer 2, so the decoder is never permitted to guess and cannot be steered into a different parser by crafted content. Fourth, the layer fails closed: a file that a pinned decoder cannot rebuild is not a plain raster image, so any decode or re-encode failure refuses it rather than passing it through unmodified.

How the layer makes neutralization measurable. A detection layer explains a refusal; this layer accepts the file and rewrites it, so it must answer a different question — what was removed, and how would anyone know? The question is not rhetorical, because every raster image returns a sanitized verdict: re-encoding is prophylactic and runs whether or not anything suspicious was present. A neutralization rate computed from that verdict alone is therefore 100% by construction — it would count an ordinary photograph as neutralized — and carries no information. Neutralization is meaningful only where something measurable was destroyed, so the destroyed quantity is measured and recorded, and the metric reported in Chapter Five is derived from that measurement rather than from the verdict.

The measurable quantity is trailing data. Every accepted raster format has a defined end: the end-of-image marker for JPEG, the terminating chunk for PNG, and the container size field for WebP. Bytes beyond that point are not image content; they are the appended payload the layer exists to remove. Their count, their offset, and a printable preview of their first bytes are reported, which turns "the file was sanitized" into a figure a reviewer can confirm in a hex editor at the stated offset. Two details keep the measurement honest. For JPEG the last end-of-image marker is used, because an embedded thumbnail is itself a complete JPEG carrying its own marker, so the first match would sit inside the metadata rather than at the true end. And where the end of image cannot be located at all, the result is flagged as undetermined rather than reported as an absence of trailing data, so a file whose structure defeated the survey is never counted as evidence that nothing was hidden. The

re-encoded output is additionally hashed, and that digest is recorded, so the determinism claim of Section 3.9.1 is checkable by re-running the same input rather than merely asserted.

```
// Layer 6 - decode to pixels, rebuild from pixels only.
// Order matters: the trailing-data survey and the bomb guards both run on the
// ORIGINAL bytes, before anything is decoded.

// 1) Bomb guards from the HEADER ALONE - no pixel decoded yet.
var info = new MagickImageInfo(context.FileStream);
if (info.Width > _options.MaxWidth || info.Height > _options.MaxHeight)
    return ScanResult.Reject(Name, "dimensions exceed limit", evidence); // CWE-409
if ((long)info.Width * info.Height > _options.MaxTotalPixels)
    return ScanResult.Reject(Name, "pixel count exceeds limit", evidence);

// 2) Trailing-data survey: the quantity that makes neutralisation MEASURABLE.
//   JPEG -> last EOI; PNG -> last IEND + CRC; WebP -> RIFF size field.
var trailing = MeasureTrailingData(original, type);
evidence.Add(DescribeTrailing(trailing, original.Length));

// 3) Decode with the format PINNED from Layer 2 - the parser never guesses.
using var image = new MagickImage(context.FileStream,
    new MagickReadSettings { Format = format });

// 4) Optional pixel-domain LSB clearing (see the text).
if (_options.ClearLeastSignificantBit)
{
    if (image.ColorType is ColorType.Palette or ColorType.PaletteAlpha)
        image.ColorType = ColorType.TrueColor; // act on intensities, not indices
    image.Evaluate(Channels.RGB, EvaluateOperator.And, 0xFE); // deterministic
}

if (_options.StripMetadata) image.Strip(); // EXIF/XMP/IPTC/ICC/comments

// 5) Re-encode from pixels only, and hash the output so determinism is checkable.
var reEncoded = new MemoryStream();
image.Write(reEncoded, format);
evidence.Add(DescribeRewrite(original, reEncoded, format, trailing)); // + SHA-256
return ScanResult.Sanitize(Name, reEncoded, evidence);
```

Listing 7: Header-only bomb guards, the trailing-data survey, pinned-format decoding, and rebuild from pixels (Layer 6).

Pixel-domain steganography. Rebuilding from pixels removes everything outside the pixel data but does not disturb the pixels themselves, so a payload hidden in the least-significant bits of the pixel values survives a lossless re-encode. For deployments that must also close that channel, the layer exposes a clearing step, controlled by the `ClearLeastSignificantBit` configuration key, which zeroes the least-significant bit of every colour channel by a bitwise-AND with `0xFE` after first normalising any palette-indexed image to true colour so that the operation acts on pixel intensities rather than on palette indices. Because each pixel is altered by at most one intensity level, the step is visually imperceptible, and it is applied uniformly to clean and to malicious files. The step is applied to every accepted raster format, but the evaluation in Section 5.6 establishes its effect for the lossless case only; on a lossy format the re-encode itself perturbs the low-order bits, so the bit-

plane measure used there does not apply. The key defaults to disabled in code so that the ablation study can measure re-encoding with and without it, and was enabled in the configuration under which the results of Section 5.6 were produced.

4.7.2 SVG Sanitization (Layer 7)

Layer 7 parses an SVG and removes its active content while preserving the vector drawing. Three mechanisms operate in sequence, and they produce two different verdicts rather than one.

The first mechanism is the parser configuration. Document-type declarations are prohibited outright and no external resolver is supplied, which closes external-entity resolution and entity expansion at the source rather than attempting to strip a declaration after accepting it. A document carrying a document-type declaration therefore fails to parse, and a parse failure refuses the file. This means an SVG carrying an external-entity declaration is refused, not sanitized — the drawing is not preserved, because a document that cannot be parsed under a safe configuration is not treated as an image at all. It also means a hostile document and a merely malformed one fail here identically, so parse failures are recorded as failures rather than as detections; counting the two together would inflate whichever figure they were added to.

The second mechanism is root validation. Type detection in Layer 2 matches the opening SVG marker anywhere in the leading bytes, including inside a comment, so a document of another type can reach this layer classified as an SVG. Requiring the root element to be an SVG element closes that path: pruning the children of a foreign root while leaving the root in place would have stored a non-image document to be served as an image.

The third mechanism is sanitization proper, and it is built on allow-lists rather than deny-lists. Only a named set of drawing elements survives, and within those, only a named set of presentational attributes; everything absent from either list is removed regardless of whether it was anticipated. This is materially stronger than removing a known-dangerous set, because it also removes constructs that were never enumerated: event-handler attributes of any name, hyperlink and cross-reference attributes and therefore every URI scheme they could carry, animation elements that can assign attribute values at runtime and so reach an attribute the allow-list would otherwise have blocked, style elements that can load external resources, and nested browsing or plugin contexts. The constructs whose removal is reported individually, because their presence is a finding rather than housekeeping, are listed in Table 13. Removal counts are reported per element and per

attribute name, so a stripped script element is not reported with the same weight as a stripped decorative element. For a document that parses, has an SVG root, and is reduced to allow-listed content, the outcome is a neutralized, accepted file: a legitimate vector graphic that merely carried a handler is preserved rather than discarded.

Table 13: SVG constructs handled by the sanitization layer, the risk each poses, and the resulting verdict.

Construct	Risk	Action	Verdict
Document-type declaration / external entity	XXE, SSRF, file disclosure, entity expansion	Parsing prohibited at the reader	Reject (parse failure)
Non-SVG root element	Non-image document stored and served as an image	Root element required to be svg	Reject
script	Stored XSS — the primary vector in SVG	Not on the element allow-list; removed	Neutralize
foreignObject	Arbitrary embedded XHTML/JS; documented sanitizer bypass	Removed	Neutralize
animate, animateTransform, set	Runtime attribute assignment, reaching blocked attributes	Removed	Neutralize
use, image	Cross-document reference and external / data: loading	Removed	Neutralize
style	CSS can load external resources	Removed	Neutralize
iframe, embed, object	Nested browsing context / plugin content	Removed	Neutralize
a, handler	Carries an href, therefore a scheme; handler element	Removed	Neutralize
Event-handler attributes (any on*)	Stored XSS	Not on the attribute allow-list; removed	Neutralize
href, xlink:href	External reference, javascript: URI	Not on the attribute allow-list; removed entirely	Neutralize
Any unenumerated element or attribute	Unknown	Not on either allow-list; removed	Neutralize
Allow-listed shapes, paths, gradients, presentational attributes	None	Preserved	—

```

// Layer 7 - SVG. Parser configuration first, then root validation, then
// allow-list pruning. Note the two different verdicts.

// 1) DTD prohibited, no resolver: XXE and entity expansion closed at the source.
//    A document carrying a DOCTYPE throws here and is REFUSED, not sanitised.
var settings = new XmlReaderSettings
{
    DtdProcessing = DtdProcessing.Prohibit,
    XmlResolver = null
};
using var reader = XmlReader.Create(context.FileStream, settings);
var doc = XDocument.Load(reader);           // throws -> Reject(SVG-PARSE)

// 2) Layer 2 matches "<svg" anywhere in the leading bytes, including inside a
//    comment, so the ROOT must be checked or a foreign document could be stored
//    and served as an image.
if (!string.Equals(doc.Root?.Name.LocalName, "svg",
    StringComparison.OrdinalIgnoreCase))
    return ScanResult.Reject(Name, $"{ReasonBadRoot}: root is not <svg>.", evidence);

// 3) ALLOW-LIST pruning, not a deny-list: anything unenumerated is removed,
//    including handlers and href/xlink:href that no deny-list anticipated.
SanitizeElement(doc.Root, report); // keeps AllowedElements / AllowedAttributes only

return ScanResult.Sanitize(Name, Serialize(doc),
    report.BuildEvidence(originalSize, cleanedSize));

```

Listing 8: DTD prohibition, root validation, and allow-list pruning, with refusal and sanitization as distinct outcomes (Layer 7).

The sanitizer, shown in Listing 8, produces the two verdicts separately: the reader configuration turns a document-type declaration into a parse failure and therefore a refusal, the root check turns a foreign document into a refusal rather than something to be pruned, and only the allow-list pruning yields a sanitized document that renders identically but can no longer execute.

4.8 Caching Subsystem

The detection layers are the most expensive stages in the pipeline, and identical files are frequently re-uploaded. A caching decorator wraps layers 5, 5b, and 5c behind a content-addressed verdict cache whose key has three parts, shown in Figure 7: the name of the wrapped layer, the SHA-256 digest of the file content, and a fingerprint of the rules file. Five design rules govern it, and each is a security property rather than an optimisation.

CachedScanLayer — three-part cache key

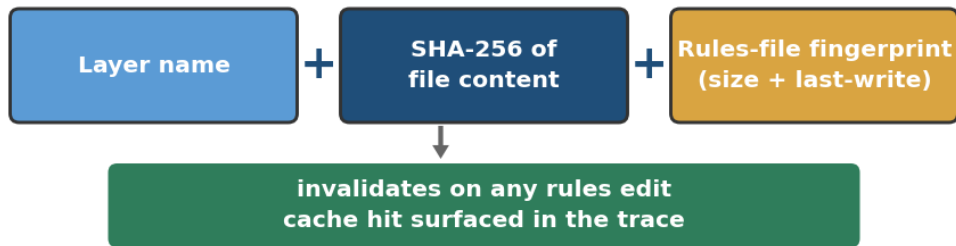


Figure 7: CachedScanLayer three-part cache key

First, only detection layers may be wrapped. A sanitization verdict carries a live stream that has been consumed, so it cannot be replayed from a cache; the constraint is enforced structurally by passing a sanitized result through uncached rather than by relying on the registration to be correct.

Second, the layers that depend on the filename — the extension whitelist, the declared-type match, and the double-extension check — are deliberately left outside the cache and run on every upload. This makes the classic bypass of submitting identical bytes under a different, malicious filename impossible by construction rather than merely unlikely: a cached verdict can never be inherited across a change of name, because the name is never part of what was cached.

Third, the content digest is SHA-256 rather than a shorter hash. A collision-capable digest would itself become the bypass: an attacker could arrange for a benign colliding file to be cached first and then upload the payload under the same key.

Fourth, the rules fingerprint binds a cached verdict to the rule set that produced it. The fingerprint combines the size of the rules file with its last-write timestamp, read on every scan at the cost of two inexpensive filesystem calls; any edit to the rules therefore yields a different key for every file and retires every stale verdict instantly, with no manual flush. A content digest of the rules file would be a stronger binding and is noted in Section 6.8 as a limitation, since a modification that preserved both size and timestamp would not invalidate the cache. Where the fingerprint cannot be read at all, a unique value is substituted, which disables caching for that single scan rather than risking a stale verdict under an unknown rule set — failing toward correctness rather than toward speed.

Fifth, the cache is bounded in both size and time: entries carry unit cost against a configured maximum entry count and expire after a configured interval, so a flood of unique uploads cannot turn the cache into a memory-exhaustion vector.

A repeated upload of an identical file returns the stored verdict and drops the scan cost to near zero, and the cached result is returned as a copy marked as a cache hit rather than by mutating the stored entry, since that entry is shared by every future hit on the same key. The hit is surfaced in the trace so a fast verdict is never mistaken for a skipped one. Finally, the SHA-256 used here is a cache key and nothing more: the system holds no database of known-malicious digests, and detection comes from the layers themselves rather than from matching a stored list.

```
// The three-part key. Content digest + wrapped-layer name + rules fingerprint.
var key = $"{_inner.Name}|{context.ContentHash}|{GetRulesFingerprint()}";

// Size + last-write time, read on every scan: two cheap syscalls, so a rule edit
// takes effect immediately. A content digest would bind more tightly - see 6.8.
private string GetRulesFingerprint()
{
    if (_rulesFilePath is null) return "none";
    try
    {
        var fi = new FileInfo(_rulesFilePath);
        return fi.Exists ? $"{fi.Length}-{fi.LastWriteTimeUtc.Ticks}" : "missing";
    }
    catch
    {
        // Fail toward correctness: a unique value disables caching for THIS scan
        // rather than risking a stale verdict under an unknown rules state.
        return Guid.NewGuid().ToString("N");
    }
}

// A sanitization verdict holds a consumed stream and is NEVER cached.
if (result.Decision != ScanDecision.Sanitized)
    _cache.Set(key, result, new MemoryCacheEntryOptions
    {
        Size = 1, // bounded by SizeLimit
        AbsoluteExpirationRelativeToNow = _ttl
    });
```

Listing 9: The three-part cache key and the size-plus-timestamp rules fingerprint, with sanitization results never cached.

Listing 9 shows how the three-part key is formed. Because the rules fingerprint is part of the key, any edit to the rules produces a different key for every file, which is exactly what makes a stale cached verdict impossible after a rule change; and because the fingerprint is size plus last-write time rather than a digest of the rule contents, the binding is cheap on every scan but weaker than a content digest, which is recorded as a limitation in Section 6.8.

4.9 Configuration Model

The pipeline's behavior is governed by a set of runtime configuration sections, allowing thresholds, depths, and modes to be tuned without code changes. The principal sections are listed in Table 14.

Section	Governs
Upload	Accepted extensions, accepted declared types, and the application size limit
Yara	Rule-set path and scanning options
PdfDecode	Inflation depth, expansion ratio, and the per-stream and total byte caps
ReEncoding	Dimension and pixel limits, JPEG quality, metadata stripping, and least-significant-bit clearing
Cache	Cache behavior and invalidation
Demo	Comparison endpoints on / off (server-side switch)

Table 14: Runtime configuration sections exposed by the pipeline.

A server-side demo switch gates the comparison endpoints used for evaluation; when it is off, those endpoints return a not-found rather than a forbidden response, so their existence is not disclosed in production. This switch must be set off before deployment, and treating it as configuration rather than a compile-time constant keeps that decision explicit. One qualification belongs here: the endpoints gated in this way are the comparison and corpus endpoints. A separate status endpoint reports the state of the switch itself and answers on every request, so the existence of the evaluation subsystem is not concealed by that endpoint even when the switch is off.

4.10 Security-Headers Middleware

Independently of the inspection layers, a middleware component sets a baseline set of response headers on every response, hardening the consumer side of the upload lifecycle as the pipeline hardens the acceptance side. Six controls are applied. Content-type sniffing is disabled, so a browser cannot reinterpret a response away from its declared type — the mechanism by which a stored file with a benign declared type can still be executed as script. Framing is refused outright, closing clickjacking against the interface. The referring URL is withheld from other origins. Browser capabilities the application does not use — camera, microphone, and geolocation — are switched off. Cross-origin loading of resources served by the interface is restricted to the same origin. And a content-security policy constrains what a served document may load and do: the default source and the base URI are restricted to the origin, plugin objects are forbidden entirely, framing ancestors are forbidden, and image sources are limited to the origin and inline data.

One relaxation is disclosed rather than left implicit: inline script and style are permitted in the policy so that the development-only interface documentation continues to function. That documentation is disabled outside development, and tightening the policy for a production deployment is a required step alongside disabling the demonstration switch. The server also suppresses its own identifying response header, so the runtime is not advertised.

These headers complement neutralization rather than duplicating it. The pipeline removes active content from a file before it is stored; the headers reduce the impact of anything that survives, and constrain how even a fully clean file is interpreted once served.

4.11 Verdicts and the Evidence Trail

Every file resolves to one of three verdicts: refused, rewritten and accepted, or accepted unmodified. Regardless of verdict, the pipeline records a per-layer trace with wall-clock timings for every layer that ran — not only for the layer that acted — so that for any file it is possible to establish which layers executed, which one acted, and on what basis.

The trace carries more than prose. Each entry holds a list of typed observations, and each observation records a machine-readable class, the thing observed, the observed value in readable form, the byte offset where the observation has a position, the interpretation strength, and an external citation where one applies. Three properties of that structure matter. The interpretation strength is sourced from the rule or check that produced the observation and is never invented by the layer that reports it, which is what allows an acceptance policy to be defined over severities at all. Observations are emitted on acceptance as well as on refusal, because a layer that speaks only when it refuses leaves no way to show what it measured on the files it passed — and the trailing-data survey of Section 4.7.1 is precisely such a measurement on an accepted file. And the observation class is a stable identifier rather than a sentence, so corpus results can be tallied by cause instead of by matching prose that changes whenever the wording does.

A note on what a refusal asserts. A static filter cannot establish that a payload is malicious; establishing that requires execution or semantic analysis, both outside the remit of an upload gate. What the pipeline can establish is one of three things, and the recorded observations are what distinguish them: that the file matches a documented exploit trigger or signature; that the file carries a capability to act without user interaction; or that the file shows structural signs of

concealing its own content. Refusals are therefore stated as policy outcomes — that a given capability is not permitted on this surface — rather than as verdicts of malice. The distinction is not cosmetic: it is what makes the false-positive rate reported in Chapter Five a meaningful quantity rather than a count of disagreements about intent.

4.11.1 Reading a Trace

A trace is read top to bottom as the file's journey through the pipeline. Each entry names a layer, the decision it returned — accepted, rejected, or sanitized — and, where relevant, the evidence and the time the layer took. For an accepted-clean file every entry is a pass; for a rejected file the trace passes through the early layers and terminates at the layer that rejected, with the reason recorded there and no later layers run, because rejection short-circuits the remainder; for a neutralized file the trace shows a transformation layer rewriting the bytes, with the before-and-after sizes recorded, and the file continuing to acceptance.

Two fields are especially useful in practice. The acting-layer field names the single layer responsible for the verdict, so a reviewer can establish why a file was refused or rewritten without reading the whole trace. The cache flag records whether a detection result was replayed from the verdict cache rather than freshly computed, so a fast verdict is never mistaken for a skipped one.

4.12 The Instrument-Panel Frontend

The decisions of the pipeline are inspected through a React frontend styled as a dark instrument panel. It renders the per-layer trace as a pipeline rail that stops at the acting layer, colors the verdict in one of three ways — accepted, rejected, or neutralized — and, in evaluation mode, presents a comparison panel that shows the verdicts of all six validators — five reference implementations and the layered filter — on the same file side by side. The frontend is not merely cosmetic: by making the evidence trail visible it turns the transparency principle into something an analyst can actually use, and it is the surface on which the FromCache flag and the per-layer timings are read.

4.13 Worked Example: A File's Journey Through the Pipeline

To make the interaction of the layers concrete, consider a single crafted input: a file named `photo.png` that carries a genuine PNG header, valid pixel data, and — hidden in an ancillary text chunk — a small PHP web shell. Tracing this file through the pipeline shows how the layers combine to produce a safe outcome that no single layer could reach alone.

At Layer 1 the extension .png is on the allow-list, so the file passes. At Layer 2 the leading bytes are the genuine PNG signature, so magic-byte validation also passes — this is the point at which a magic-byte-only validator would accept the file and stop. At Layer 3 the declared content type image/png agrees with the type detected from the bytes, so declared-versus-detected matching passes — it has no visibility into the file's chunk structure or its length — and at Layer 4 the single, legitimate extension raises no double-extension flag. The four structural layers, in other words, all pass, exactly as the attacker intended when they built a structurally valid PNG.

At Layer 5 the YARA scan inspects the file's bytes and may match the PHP web-shell pattern in the text chunk; if the payload is obfuscated enough to slip the signatures, the file still carries it forward. Because the file is a PNG rather than a PDF, the PDF-specific layers 5b and 5c pass it through. The decisive moment is Layer 6: the image is decoded to its pixels and re-encoded from scratch, and the text chunk — which is not pixel data — is simply not part of the rebuilt file. The web shell is gone. The verdict is neutralized-and-accepted, the file remains a valid, viewable PNG, and the per-layer trace records that image re-encoding was the acting layer.

The lesson of the walkthrough is that the file defeated every structural check and possibly the signature scan, yet was still rendered harmless, because the transformation layer removed the payload by construction rather than by recognizing it. This is defense in depth working as intended: the attacker satisfied the properties the early layers check, but could not satisfy the one the transformation layer enforces — that the accepted file consist of nothing but its pixels.

4.14 Chapter Summary

SecureUploader composes nine independent layers — four structural, three detection, and two transformation — behind a single IScanLayer abstraction, in order of increasing cost and specificity, wraps the detection layers in a rules-aware content cache, hardens served responses with security headers, exposes its behavior through configuration, and records an auditable per-layer evidence trail rendered by an instrument-panel frontend. Chapter Five evaluates this design empirically.

Chapter Five — Experimental Results

5.1 Introduction

This chapter evaluates SecureUploader along two axes. The first is aggregate detection performance on a large, realistic public corpus, reported as a confusion matrix and standard metrics with an analysis of both the false positives and the false negatives. The second is evasion resistance on a hand-built adversarial corpus, where the pipeline is compared, fixture by fixture, against five reference validators, followed by a per-sample analysis of the most instructive cases, a note on timing, and a contextual comparison with machine-learning detectors and related systems.

5.2 Evaluation Setup

Aggregate performance is measured on the full CIC-Evasive-PDFMal2022 corpus of 10,012 files — 5,549 malicious and 4,463 clean PDF documents. Evasion resistance is measured on the adversarial corpus described in Chapter Three, comprising real polyglots, crafted inert fixtures, and evasive documents whose expected behavior is fully known. Metrics follow the definitions in Chapter Three, and every verdict is produced by the same pipeline configuration and under the acceptance policy stated in Section 5.3.1, so that results are directly comparable.

5.3 Aggregate Results

5.3.1 Confusion Matrix and Metrics

The figures in this section are reported under the severity-gated acceptance policy — refusing only high- and critical-severity findings — which is defined and analysed in Section 5.3.3. The as-implemented policy, under which the signature layers refuse on any match irrespective of declared severity, is reported alongside it there.

On the full 10,012-file corpus the pipeline correctly classified 5,298 of 5,549 malicious files and 4,067 of 4,463 clean files, giving 95.48% recall, 93.05% precision, 93.54% accuracy, and a 94.25% F1 score, with a specificity of 91.13%. The confusion matrix and the derived metrics are shown in Figure 8, and the four headline metrics are shown together in Figure 9.

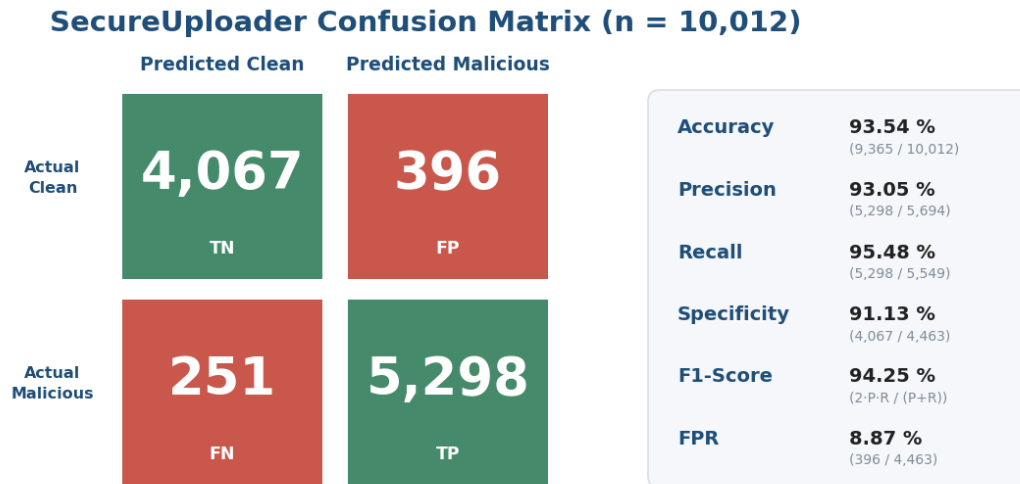


Figure 8: Confusion matrix and performance metrics

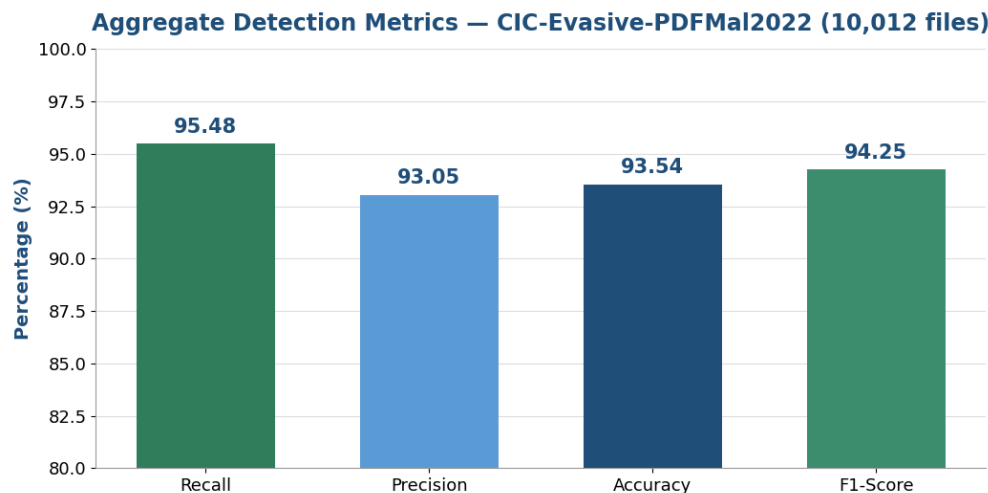


Figure 9: Aggregate detection metrics

These results place the pipeline in a strong position for a training-free, byte-level system. Recall above 95% means fewer than five malicious files in a hundred pass undetected, and a sub-9% false-positive rate on a corpus whose clean half was engineered to resemble the malicious class is low enough to be workable. That the four metrics are close together indicates the pipeline is not trading one against another — it is neither over-aggressive nor over-permissive — which is the balance a deployable validator needs.

5.3.2 Interpretation

It is worth reading the numbers against the operating regime. The pipeline achieves this without any training phase and without extracting a feature vector: it acts on raw bytes and reaches a verdict through a fixed sequence of layers. A recall of 95.48% on a corpus explicitly built to include evasive PDFs indicates that the detection layers, and in particular the recursive inflation that exposes concealed payloads, are doing substantive work rather than catching only the easy cases.

5.3.3 False-Positive Analysis

The false-positive result must be read against both the construction of the clean set and the acceptance policy under which it was measured. The pipeline as implemented refuses a file on any signature match, irrespective of the severity the matching rule declares. Under that policy, and on a corpus whose benign half was selected to resemble the malicious class, medium-severity signature and structural findings fire on most clean documents: the full-corpus run refused 4,285 of the 4,463 clean files, a false-positive rate of 96.0%. That figure characterises the dataset at least as much as the pipeline — the clean half is adversarial by construction — but it is also directly actionable, because every finding already carries the severity declared by the rule that produced it, recorded in the `top_severity` column of the corpus results.

Re-deriving the verdicts from those recorded severities under a stricter acceptance policy — refusing only high- and critical-severity findings while logging the remainder — yields 396 false positives out of 4,463, an 8.87% false-positive rate on the same clean set, with no material change to true detections (recall moves from 95.78% to 95.48%). Figure 10 shows the two rates side by side. It should be stated explicitly that this second figure is obtained by re-analysis of the recorded per-finding severities and not by a gate implemented inside the pipeline; the severity threshold is specified here as a policy and its implementation as a configurable acceptance level is identified in Section 7.4 as an immediate next step. Reporting it this way keeps the measurement separable from the mechanism: the severities are a property of the rule set and were recorded during the run, so the derived rate is reproducible from the published results file, but the shipped pipeline would need the threshold added before it produced that rate directly.

The residual 396 false positives are dominated by encrypted PDFs that cannot be scanned and on which the pipeline fails closed, together with a smaller number of documents whose structure genuinely matches a high-severity rule and a further group refused for a declared-type

contradiction arising from a benign renaming (Section 2.8.3). The first group is addressed by downgrading the encrypted-PDF rule; the second is the irreducible cost of a fail-closed posture on a corpus designed to blur the two classes; the third is a policy question about how strictly a misnamed upload should be treated.

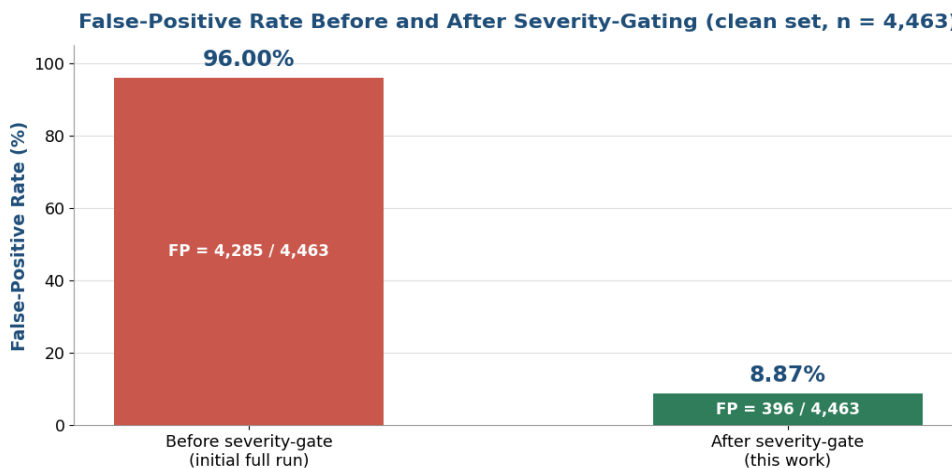


Figure 10: False-positive rate before and after severity-gating

5.3.4 False-Negative Analysis

The 251 malicious files that passed undetected — 4.5% of the malicious set — are the complement worth examining. They correspond to malicious PDFs whose indicators fall outside the current rule set and are not exposed by inflation — payloads that neither match a signature nor sit behind a FlateDecode chain the layer reaches. The corpus results support that reading directly: every one of the 251 carries the uncoded reason code, meaning each passed through all nine layers without any layer reaching a stopping condition, rather than being lost to a scanner error or a truncated inspection. This is the expected failure mode of a signature-and-structure approach with no learned component: it catches what its rules and its structural checks describe, and misses novel indicators for which no rule yet exists. Growing the rule set and the adversarial corpus, and the planned ablation study, are the mechanisms by which this residue is intended to be reduced.

5.4 Adversarial-Corpus Comparison

Aggregate accuracy on realistic documents does not by itself demonstrate evasion resistance, because a corpus of real-world files contains few inputs engineered specifically to bypass a given check. The adversarial corpus fills that gap. Each fixture is run through six validators: the four DVWA reference levels, re-implemented in C# from their documented logic, a baseline validator, and the full layered pipeline. The documented logic of each is summarised in Table 15.

Comparability disclosure. Three properties of the reference validators bound what the comparison measures, and results should not be read without them. First, the reference levels accept only JPEG and PNG by extension and by declared type, and enforce a size limit of 100,000 bytes at every level above the weakest. Any fixture in another accepted format, or above that size, is therefore refused by those validators on type or size grounds rather than by detecting anything — a refusal that keeps the file out but demonstrates no inspection capability. Those refusals are marked distinctly in Figure 11 so they are not read as detections, and the detection rates in Figure 12 are computed over the subset of fixtures inside the intersection of accepted types and sizes, which is the only subset on which the comparison measures inspection rather than exclusion. Second, the baseline validator checks file size, the final filename extension, and the declared content type; it performs no signature validation of the leading bytes, so a payload renamed to an accepted extension and submitted with the matching declared type passes it. Third, the reference levels model the image-header probe of the original with a header-only inspection from the project's own imaging library, re-encode with that library rather than with the original's imaging extension, and inherit process-wide memory ceilings that the original does not have. They are therefore marginally more robust than a live deployment of the reference application against decompression bombs, and the comparison should be read as conservative rather than exact. The reference application itself was not executed; only its documented validation logic was reproduced, so the comparison isolates validation behaviour from any incidental property of a running deployment.

Scanner	Reproduced validation logic	Accepted types / size	Key weakness
DVWA Low	No validation	any	Accepts anything
DVWA Medium	Declared content type + size	image/jpeg, image/png; ≤ 100 KB	Trusts attacker-supplied declared type
DVWA High	Final extension + size + image-header probe	jpg, jpeg, png; ≤ 100 KB	Header probe reads the header only; appended-payload images pass intact
DVWA Impossible	Extension + size + declared type + header probe + image re-encode	jpg, jpeg, png; ≤ 100 KB	No handling of vector or document formats
BasicValidation (baseline)	Size + final extension + declared content type	project allow-list	No signature validation of the leading bytes; a renamed payload with a matching declared type passes
SecureUploader	Nine layers + neutralization	project allow-list; ≤ 5 MB	This work

Table 15: The six validators, the logic each reproduces, and the types and sizes each will consider at all.

Figure 11 shows, fixture by fixture, which validators blocked each input by inspecting it, which merely refused it because its type or size was outside what they accept, and which accepted it unsafely; Figure 12 summarises the detection rates over the fixtures that every validator was willing to consider.

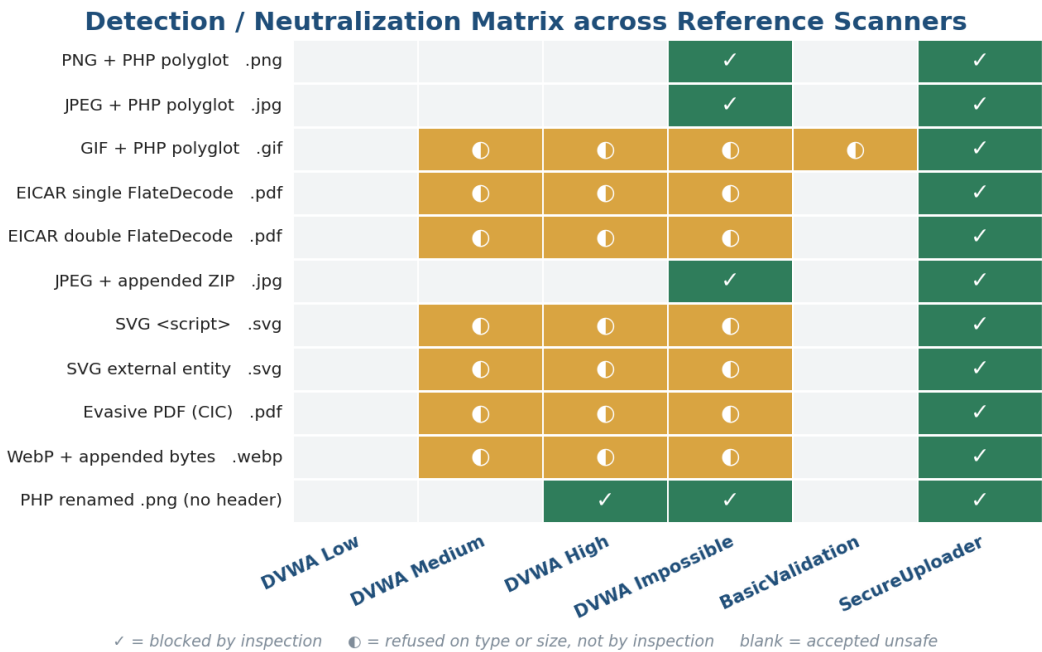


Figure 11: Detection / neutralization matrix, with type-or-size refusals marked separately

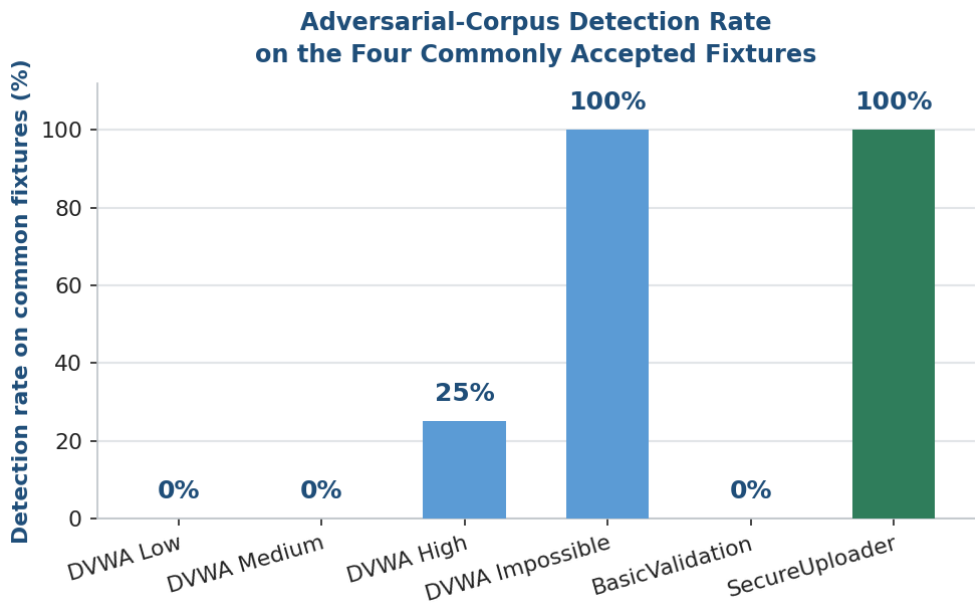


Figure 12: Adversarial-corpus detection rate on the commonly accepted fixtures

The pattern is one of coverage rather than of hit rate, and stating it that way is both more accurate and stronger. On the four fixtures that every validator accepts by type and size — the JPEG and PNG polyglots under the size limit — the strongest reference level, which re-encodes images, neutralizes all four, exactly as this work does. That agreement is expected rather than disappointing: re-encoding is the correct answer to an image polyglot, and the fixture confirms the theory. The difference lies elsewhere. Every remaining fixture is in a format the reference levels do not accept at all — a PDF, an SVG, a WebP, or a GIF — so their refusals are exclusions on type, not detections, and they have no answer of any kind to a payload behind nested filters, to active content in a vector image, or to bytes appended to a WebP container. The baseline, which validates size, extension, and declared type but never the leading bytes, accepts ten of the eleven fixtures outright, including a PHP payload renamed to an image extension. SecureUploader blocks all eleven by inspection, because each evasion class has a dedicated layer across the whole accepted type set — which is precisely the qualitative claim of Figure 16, now measured. The verdicts for the representative fixtures are given in Table 16.

Fixture	SecureUploader verdict	Acting layer
PNG + PHP polyglot	Neutralized	Image re-encoding (6)
JPEG + appended ZIP	Neutralized	Image re-encoding (6)
EICAR single FlateDecode	Rejected	PdfFlateDecode (5b), depth 1
EICAR double FlateDecode	Rejected	PdfFlateDecode (5b), depth 2
SVG script / onload	Neutralized	SVG sanitization (7)
SVG foreignObject	Neutralized	SVG sanitization (7)
SVG with external entity	Rejected	SVG sanitization (7), DTD prohibited
PHP renamed .png (no header)	Rejected	MagicBytes (2)

Table 16: Per-fixture verdict of SecureUploader on representative adversarial samples.

5.5 Per-Sample Analysis

Four fixtures are examined in detail because each isolates a specific contribution of the pipeline and together they cover rejection, neutralization, and caching.

5.5.1 EICAR Behind a Single FlateDecode

A PDF carrying the EICAR test string inside a single FlateDecode stream passes plain signature scanning — the raw stored bytes do not contain the marker — but is rejected by Layer 5b after one level of inflation exposes the payload. This is the minimal demonstration that inflation must precede scanning: the very same payload is invisible before inflation and unmistakable after it, so the order of the two operations is the whole point.

5.5.2 EICAR Behind a Double FlateDecode

Wrapping the same payload in two FlateDecode passes defeats any scanner that inflates only once. Layer 5b's recursive inflation reaches the payload at the second level and rejects the file, confirming that the fix generalizes to nested rather than merely single filters. The pair of fixtures together is the ablation argument for the layer: remove recursive inflation and the double-wrapped payload passes; keep it and both are caught.

5.5.3 Appended-Payload Neutralization

A JPEG with an appended payload is not rejected but neutralized. Decoding the image to pixels and re-encoding it discards the appended bytes deterministically, reducing the file from 28,554 to 14,863 bytes as shown in Figure 13, and the same input always yields the same output. The figure is not merely the difference between two file sizes: the removed region is located by the trailing-data survey of Section 4.7.1, which reports the offset of the last end-of-image marker together with a printable preview of the bytes beyond it, so the measurement can be confirmed in a hex editor at the stated offset. The visible image is preserved, so the neutralized file remains useful — the defining property of neutralization as distinct from rejection, and the reason a legitimate image that merely carried extra bytes is not lost to the user.

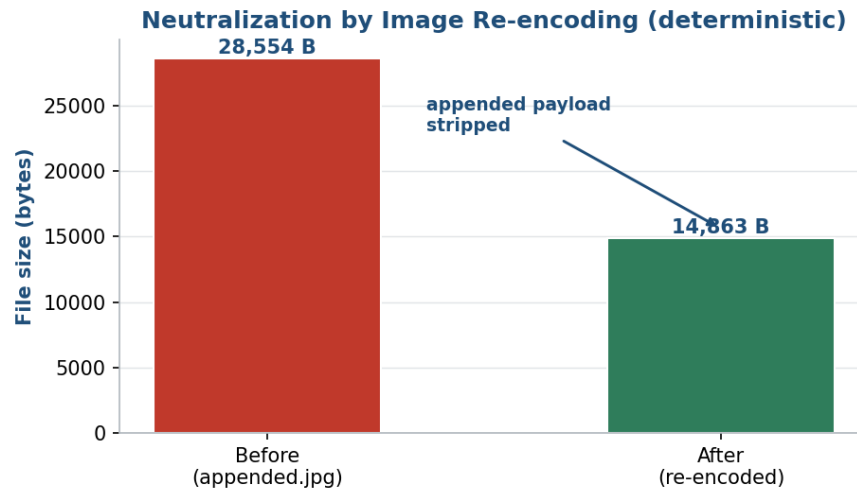


Figure 13: Neutralization by image re-encoding

5.5.4 Cache Behavior

Re-uploading an identical file returns the cached verdict and drops the scan cost to near zero, tagged in the interface as a cache hit. Because the cache key includes the YARA rules fingerprint, editing the rule set invalidates the entry immediately, so caching accelerates repeated scans without ever masking a newly-added detection — the correctness property that the stale-cache finding in Chapter Six exists to guarantee.

5.5.5 PNG Text-Chunk Parasite

A PNG carrying a payload inside an ancillary text chunk is a parasite polyglot: the payload lives in a metadata region that the image renderer ignores but that a second parser can be made to act on. Plain rendering never touches the chunk, so a validator that only confirms the image displays correctly is satisfied. The re-encoding layer, however, rebuilds the file from decoded pixels and strips ancillary chunks, so the parasite is removed and the file is neutralized — the same mechanism that defeats appended payloads applied to an interior metadata region rather than a trailing one.

5.5.6 WebP with Appended Bytes

WebP was added to the accepted set in place of GIF, and it is subject to the same appended-payload evasion as JPEG and PNG. A WebP with bytes appended after its valid image data is neutralized by decoding and re-encoding, which discards everything outside the pixel data. The end of the image is located for this format from the container size field in bytes four through seven, which is what makes the appended region measurable rather than merely presumed. Including WebP in the

fixtures confirms that the re-encoding defense is not specific to a single raster format but applies uniformly across the accepted raster types, which is the property that lets a single transformation layer cover an entire class of inputs.

5.5.7 End-to-End Walkthrough of a Neutralized Upload

It is instructive to follow one file through the entire pipeline. Take the JPEG-plus-appended-ZIP polyglot of the worked example in Chapter Two, submitted to the endpoint under the name `photo.jpg`. The extension-whitelist layer accepts the extension; the magic-byte layer confirms the leading bytes are a valid JPEG header and records `jpeg` as the detected type; the declared-versus-detected layer finds `image/jpeg` consistent with that detection and passes the file, having no visibility into the file's length or its trailing bytes; and the double-extension layer finds no concealed interior extension. The four structural layers therefore all pass, exactly as the attacker intended when constructing a structurally valid JPEG.

The file then reaches the detection layers. Signature scanning finds no rule match in the raw bytes; the `FlateDecode` layer does not apply, because the file is not a PDF; and embedded-link inspection likewise does not apply. The file arrives at the image re-encoding layer still carrying its appended archive. There it is decoded to pixels and re-encoded from scratch, and the appended ZIP — being no part of the pixel data — is discarded. The pipeline returns a neutralized verdict, storing the rewritten image and recording a trace whose acting layer is image re-encoding and whose reason notes the size reduction.

The walkthrough illustrates the division of labor the design intends. The structural layers narrowed and characterized the file without rejecting a possibly-legitimate image; the detection layers found nothing to reject on, correctly, because the payload was inert appended data rather than a known signature; and the transformation layer resolved the ambiguity by rewriting the file into a provably inert form. No single layer handled the file alone, and the verdict is fully explained by the trace — which is the behavior the whole architecture exists to produce.

5.6 Neutralization of Pixel-Domain Steganography (LSB)

The image re-encoding layer removes payloads that live outside the pixel data — appended trailers, ancillary metadata chunks, and parasite polyglots — by rebuilding the file from its decoded pixels. One class of payload is not outside the pixels, however: least-significant-bit (LSB) steganography hides data in the low-order bit of the pixel values themselves. On a lossless format such as PNG,

decoding to pixels and re-encoding preserves those values exactly, so a plain re-encode leaves an LSB payload intact. This section evaluates a dedicated countermeasure — a deterministic least-significant-bit clearing step — against a purpose-built steganography dataset, and reports it as a neutralization result rather than a detection one. The threat it addresses sits at the edge of the upload-safety model: an LSB payload is a data-exfiltration channel rather than a code-execution vector, which is why the step is offered as a defense-in-depth option rather than a core gate.

5.6.1 Dataset and Metric

The evaluation uses the classified release of the Stego-Images-Dataset introduced by Cassavia et al. (JOWUA, 2022), which consists of 512x512 PNG images carrying payloads embedded by LSB substitution across several payload types: JavaScript, HTML, PowerShell, and Ethereum addresses. From this dataset a reproducible sample was drawn with a fixed random seed, taking fifteen images from each of five classes — the four malicious payload types plus a clean control class — for a total of seventy-five images. The dataset and the tested sample are summarized in Table 17.

The metric is deliberately not a confusion matrix. A confusion matrix measures detection — acceptance against refusal — but an LSB payload is not expected to be refused; it is expected to be accepted and rewritten, exactly as in the appended-payload and parasite cases of the previous section. Nor can the metric be derived from the sanitization verdict itself: as Section 4.7.1 establishes, every accepted raster image returns a sanitized verdict because re-encoding is prophylactic, so a rate computed from that verdict would be 100% by construction and would count an unmodified photograph as neutralized. The metric is therefore defined on the payload-carrying classes only, and the clean class serves as a control on which the question is acceptance rather than neutralization. Neutralization is measured by comparing the least-significant-bit plane of each image before and after processing: an image is counted as neutralized when at least 2% of least-significant bits change, and as survived when fewer than 1% change. The threshold is generous by design, since a payload without error correction is destroyed by a single flipped bit in its compressed stream and a change of around 2% corrupts any textual header or terminator, so any measurable disturbance of the bit plane is sufficient to break a recoverable payload. The bit-plane comparison and the distortion measurement were performed by a separate measurement script rather than by the pipeline, since the pipeline's own neutralization indicator is keyed on

destroyed trailing data and cannot observe a change confined to the pixel values; the bit-plane comparison and the peak-signal-to-noise measurement were therefore performed by a separate script written for the purpose.

Property	Value
Dataset	Stego-Images-Dataset (classified release)
Reference	Cassavia, Caviglione, Guarascio, Manco, Zuppelli — JOWUA, Vol. 13, No. 3, 2022
Content	512x512 PNG images with LSB-embedded payloads
Payload types	JavaScript, HTML, PowerShell, Ethereum addresses
Tested sample	15 images per class x 5 classes (js, html, ps, eth, clean) = 75 images
Selection	Reproducible random sample (fixed seed)

Table 17: The pixel-domain steganography dataset and the tested sample.

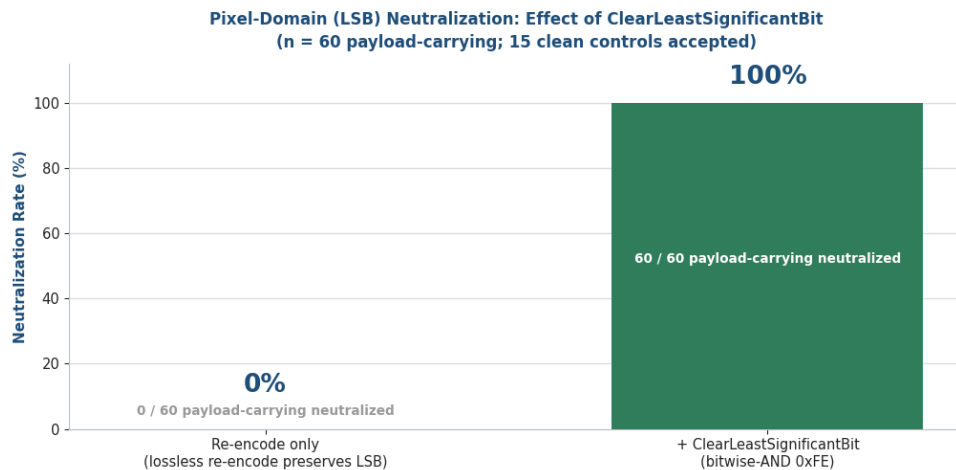
5.6.2 Results

With the clearing step enabled, every payload-carrying image in the sample was neutralized: all sixty images across the four payload classes had their least-significant-bit plane disturbed well beyond the threshold, giving a neutralization rate of 100% on the malicious classes. All fifteen images of the clean control class were accepted, as required — a clean image carries nothing to neutralize, and counting the change to its bit plane as a neutralization would reproduce exactly the measurement error the metric was defined to avoid.

The contrast with a plain re-encode is decisive. Before the clearing step was introduced — that is, with image re-encoding alone — none of the sixty payload-carrying images was neutralized, because lossless re-encoding of the format reproduces the pixel values bit for bit and therefore preserves the embedded payload intact. After the clearing step, all sixty are neutralized. Figure 14 shows the two conditions side by side, and the per-class breakdown is given in Table 18.

The transformation is visually imperceptible. Because only the least-significant bit is affected, each pixel changes by at most one intensity level; the measured peak signal-to-noise ratio was no lower than 49.35 dB across the sample, with a mean of 52.99 dB — comfortably above the level of approximately 48 dB commonly taken as the threshold of visual imperceptibility. The step therefore destroys the hidden channel while leaving the visible image indistinguishable from the original, which is the property that makes it acceptable to apply to every accepted image rather than only to suspected ones.

Figure 14: Pixel-domain (LSB) neutralization before and after LSB clearing (n = 60 payload-carrying; 15 clean controls accepted)



Class	n	Neutralized	Survived	Rate
js (JavaScript)	15	15	0	100%
html	15	15	0	100%
ps (PowerShell)	15	15	0	100%
eth (Ethereum)	15	15	0	100%
Total malicious	60	60	0	100%
clean (control)	15	— (all accepted)	—	15/15 accepted

Table 18: Per-class neutralization of LSB payloads on the tested sample.

5.6.3 Technique and Scope

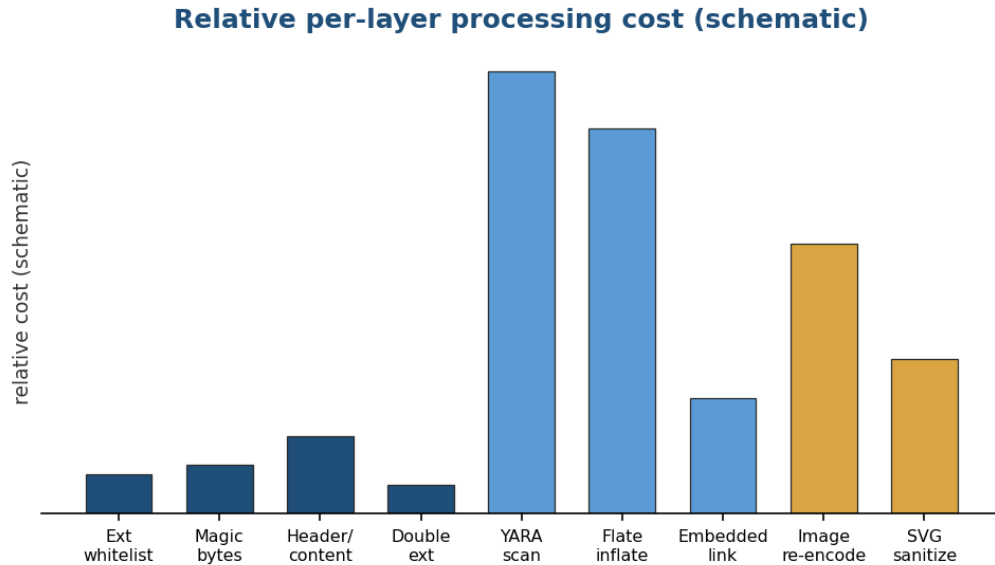
The clearing step zeroes the least-significant bit of every colour channel by a bitwise-AND with 0xFE, after first normalizing any palette-indexed image to true colour so that the operation acts on pixel intensities rather than palette indices. An earlier attempt used a posterization operator, which

rounds each channel to the nearest of a reduced set of levels; that approach is not deterministic in its effect on the low bit, because rounding can leave the least-significant bit set on light pixels, and it failed to neutralize one image in testing while succeeding on the rest by coincidence. The bitwise-AND is deterministic: it clears the low bit of every pixel without exception, which is the behavior a neutralization guarantee requires.

The scope of the operation and the scope of the result differ, and both are stated. The clearing step is applied to every accepted raster format, not only to PNG; but the evaluation establishes its effect for the lossless case alone, because on a lossy format the re-encode itself perturbs the low-order bits and the bit-plane measure used here therefore does not apply. The scope of this result is accordingly the lossless raster pixel domain. Steganography that hides data in the frequency domain — for example in the quantized DCT coefficients of a JPEG — is not addressed by clearing a spatial-domain least-significant bit, and is left as future work. This bounds the claim precisely: the layer neutralizes spatial-domain LSB payloads in lossless raster images completely and imperceptibly, and does not claim to defeat transform-domain steganography. Consistent with its status as a boundary case of the upload-safety model, the step is exposed behind a configuration flag and applied as a defense-in-depth measure rather than as one of the core rejection gates.

5.7 Timing and Performance

The per-layer trace records wall-clock timings for every stage, which makes the cost structure of the pipeline visible. The structural layers are effectively free; the dominant costs are the YARA scan and, for PDFs, the recursive inflation, with image re-encoding a moderate cost that is paid only for accepted images. Ordering the layers by cost means that most rejected files never reach the expensive stages, and the content cache means that repeated uploads of the same file skip the detection layers entirely. Together these two design choices keep the amortized cost of the pipeline low even though its worst-case per-file work is dominated by the detection layers. The relative cost of the layers is depicted schematically in Figure 15.



Structural layers are near-free; the YARA scan and recursive inflation dominate; re-encoding is moderate.

Figure 15: Relative per-layer processing cost

5.8 Contextual Comparison with Machine-Learning Detectors

Machine-learning detectors report very high accuracy on CIC-Evasive-PDFMal2022, but the comparison with SecureUploader is contextual rather than head-to-head: those systems train on pre-extracted feature tables with train/test splits, whereas SecureUploader operates on raw bytes with no training. Table 19 places the numbers side by side while making the difference in operating regime explicit.

Approach	Reported accuracy	Operating regime
ML classifier (Issakhani et al.)	~98.7%	Trained on extracted features
ML classifier (Al-Haija)	~98.84%	Trained on extracted features
SecureUploader (this work)	93.54%	Raw bytes, no training, byte-level

Table 19: Contextual comparison with machine-learning detectors on CIC-Evasive-PDFMal2022.

The point of the comparison is not to claim parity on a leaderboard but to situate a transparent, training-free, byte-level pipeline against opaque, trained, feature-level classifiers. SecureUploader

trades a few points of headline accuracy for the ability to act at the point of upload, on raw bytes, with an auditable per-layer explanation and a neutralization option that a classifier does not provide. For a deployment that must justify each decision and preserve legitimate files, those properties can matter more than the last few points of accuracy.

5.9 Comparison with Related Systems

Against the closest published analog, FileUploadChecker, SecureUploader differs in two measured respects: it adds recursive inflation and re-encoding that harden it against the nested-filter and polyglot fixtures on which a conformance-only checker would pass the file, and it reports neutralization as a distinct verdict rather than a binary decision. One qualification belongs with that claim: as a verdict, neutralization is reported directly, but as a measurement it is conditioned on a measurable destroyed quantity rather than on the verdict itself, for the reason set out in Section 4.7.1. Against the FUEL taxonomy, the adversarial corpus is organized so that its fixtures instantiate FUEL's evasion categories, so the fixture-by-fixture results in Figure 11 can be read as coverage of that taxonomy rather than of an ad-hoc set of cases.

5.10 Chapter Summary

On realistic documents the pipeline reaches 95.48% recall and 93.54% accuracy, with the false positives explained by a corpus whose clean half was engineered to resemble the malicious class and shown to collapse from a 96.0% to an 8.87% rate once a severity gate is applied and the false negatives explained by the limits of a signature-and-structure approach. On adversarial fixtures it is the only validator among six that rejects or neutralizes every input, because each evasion class has a dedicated layer. The per-sample analysis isolates the contribution of recursive inflation, image re-encoding, and caching; the timing discussion explains why the amortized cost stays low; and the contextual comparisons clarify what the pipeline trades and what it gains relative to learned detectors and to the closest published system.

Chapter Six — Discussion

6.1 Introduction

This chapter interprets the results. It explains why layering resists evasion and why the ordering of the layers matters, documents the six self-bypass vulnerabilities found and fixed during development, discusses the distinction between detection and neutralization and the value of the evidence trail, compares SecureUploader qualitatively with prior approaches, and states the limitations, the threats to validity, and the ethical considerations of working with real malware.

6.2 Why Layering Resists Evasion

The adversarial results in Chapter Five follow directly from the design principle. Because each layer reasons about an independent property, an input crafted to satisfy one layer gains nothing against the others. A polyglot with a genuine image header satisfies magic-byte validation but is destroyed by re-encoding; a nested-filter PDF passes plain signature scanning but is exposed by recursive inflation; an active SVG has a valid structure but loses its script to sanitization. No single forgery satisfies all applicable layers at once, which is precisely why SecureUploader is the only validator in the comparison that handles every fixture. The comparison also shows the converse: each reference scanner is strong exactly on the properties it checks and blind everywhere else, so its coverage is the union of a few properties rather than of all of them.

6.3 The Value of Ordering

Ordering is not merely an optimization. Running the cheap structural layers first means the expensive detection and transformation layers see only well-formed files of an accepted type, so they can make format-specific assumptions safely rather than defensively re-checking structure. It also bounds the work an attacker can force the pipeline to do: a malformed or disallowed file is rejected before it can reach the inflation or re-encoding stages, so an attacker cannot cheaply drive the pipeline into its most expensive paths. The order therefore contributes to both correctness and robustness, not only to speed.

6.4 Self-Bypass Findings and Fixes

Designing each layer with its own bypass in mind surfaced six vulnerabilities in the pipeline itself during development. Each was diagnosed and fixed, and the fixes are themselves part of the contribution, because they demonstrate the adversarial mindset applied to the tool rather than only

to external attackers. All six are summarized in Table 20, and the three whose diagnosis is most instructive are discussed individually below.

Finding	Mechanism	Fix
Nested-filter evasion	Payload behind multiple FlateDecode passes escaped a single-inflation scan	Recursive, bounded inflation to configurable depth (Layer 5b)
AV / scanner race	Host AV quarantined temp payloads before YARA could read them	Route scanner temp files to one excluded directory; RTP otherwise on
Stale-cache masking	A cached verdict could hide a newly-added rule's detection	Include YARA rules fingerprint in the cache key; invalidate on edit
Unbounded inflation	Recursive inflation with no bound made the layer a denial-of-service surface of its own: a crafted stream could be peeled until memory was exhausted	Bounded depth, per-stream and total byte caps that truncate, and an expansion-ratio refusal (Section 4.6.2)
Vector root not validated	Type detection matches the opening vector marker anywhere in the leading bytes, including inside a comment, so a document of another type could reach the sanitization layer classified as an image, be pruned of children, and be stored to be served as an image	The root element is required to be a vector root; a foreign root refuses the file rather than being sanitized around
Scheme matching by substring	Dangerous URI schemes were matched by a bare substring search, so an ordinary word containing a scheme name — a heading beginning "Profile:" contains "file:" — produced a finding on a benign document	Schemes are matched only where a URI can begin: at the start of the text or after a character that cannot form part of a scheme name

Table 20: The six self-bypass vulnerabilities found during development and their fixes.

6.4.1 Nested-Filter Evasion

The first finding was that a payload wrapped in more than one FlateDecode pass escaped a scanner that inflated only once. This is the vulnerability that motivated Layer 5b's recursive, bounded inflation, and the pair of EICAR fixtures in Chapter Five is the direct evidence that the fix works at more than one level of nesting.

6.4.2 The Antivirus / Scanner Race

The second finding was environmental but no less real: the host antivirus quarantined the decompressed payload in the scanner's temporary file before YARA could read it, so genuine detections were recorded as failures. The fix — routing all scanner temporary files to a single excluded directory while leaving real-time protection on everywhere else — removed the interference without weakening host protection, and the episode is a reminder that a detector's measured performance is entangled with the environment it runs in.

6.4.3 Stale-Cache Masking

The third finding was that a cached verdict could hide the detection of a newly-added rule: after a rule was added, an identical file already in the cache would return its old, pre-rule verdict. The fix was to include a fingerprint of the YARA rules file in the cache key, so any change to the rules invalidates every cached entry immediately. This is why the cache key has three parts rather than two, and it is the property the cache-behavior fixture in Chapter Five verifies.

6.5 Detection versus Neutralization

Treating neutralization as a first-class outcome changes what the pipeline can safely accept. A file that is ambiguous but potentially legitimate — an image with appended bytes, an SVG with a stray handler — need not be rejected outright, denying a legitimate user, nor accepted as-is, admitting a risk. It can be rewritten into a provably inert form that preserves its visible content. Neutralization is therefore not a weaker form of rejection but a distinct capability, and measuring it as a second metric captures value that a detection-only account would miss. Its limits are equally important to state, and they are now measured rather than cited. Re-encoding alone destroys everything outside the pixel data but leaves a pixel-domain payload intact on a lossless format, because the round-trip reproduces the pixel values exactly; the optional bit-clearing step of Section 5.6 closes that channel in the lossless case, within the boundary stated in Section 5.6.3. Neutralization therefore reduces rather than wholly eliminates risk for the residual class — chiefly transform-domain steganography — which is why it complements, rather than replaces, the detection layers.

6.6 Transparency and Auditability

The per-layer evidence trail is more than a convenience. Because every verdict names the layer that acted and the evidence it acted on, a decision can be reproduced, audited, and explained to a stakeholder who is not a security specialist. This is the sharpest qualitative difference from a machine-learning detector: a classifier returns a score whose derivation is opaque, whereas SecureUploader returns a verdict accompanied by the exact reasoning that produced it. The mechanism behind that claim is a typed evidence record rather than free prose: each observation carries a machine-readable class, the thing observed, the observed value, a byte offset where it has one, an interpretation strength taken from the rule or check that produced it, and an external citation where one applies. Two consequences follow. Because the strength is sourced from the detecting

rule and never invented by the reporting layer, an acceptance policy can be defined over severities at all — which is exactly what Section 5.3.3 does. And because observations are emitted on acceptance as well as on refusal, the trail also shows what was measured on the files that passed, which is what allows a rewritten file to be reported with the quantity that was destroyed rather than with a bare assurance. In settings where a wrong decision must be justified — a blocked legitimate upload, or an admitted malicious one — auditability is not a luxury but a requirement.

6.7 Qualitative Comparison with Prior Approaches

Figure 16 compares SecureUploader with three families of prior approach — DVWA-style validators, a basic extension-plus-magic baseline, and machine-learning classifiers — across six structural properties rather than on benchmark accuracy. SecureUploader is the only approach that scores across the whole set: it resists polyglots, handles nested-filter PDFs, neutralizes payloads, sanitizes SVG, requires no training, and produces a transparent trace. Each prior family is strong on some axes and absent on others, which is the same pattern the adversarial fixture results show, viewed now as capabilities rather than as per-fixture verdicts.

Qualitative Comparison: SecureUploader vs Prior Approaches

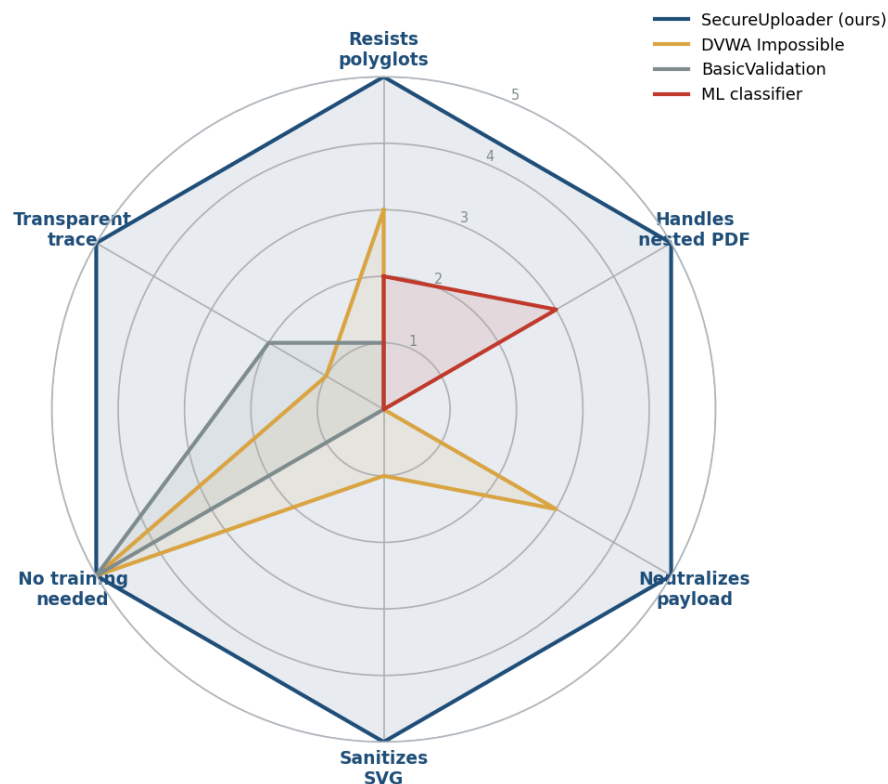


Figure 16: Qualitative comparison vs prior approaches

6.8 Limitations

- **Format scope.** The pipeline accepts a narrow set of image and PDF types; Office and archive formats, with their macro and container semantics, are out of scope and would require their own dedicated layers to handle safely.
- **Steganographic survival.** Image re-encoding alone leaves pixel-domain payloads intact on lossless formats, since the round-trip reproduces the pixel values exactly. The optional least-significant-bit clearing step evaluated in Section 5.6 neutralizes every payload-carrying image in the tested lossless sample while remaining visually imperceptible. Two residual limits are stated. First, the result is established for the lossless raster case: the clearing step is applied to every accepted raster format, but on a lossy format the re-encode itself perturbs the low-order bits, so the bit-plane measure used in Section 5.6 does not apply there and no equivalent claim is made. Second, steganography that hides data in the frequency domain — for example in the quantized transform coefficients of a lossy image — is not addressed by clearing a spatial-domain least-significant bit and is left for future work.
- **Encrypted PDFs.** Under the severity-threshold policy re-derived in Section 5.3.3, the false-positive rate on the clean set falls from 96.0% to 8.87%, and encrypted, unscannable PDFs on which the pipeline fails closed account for most of what remains; a further downgrade of the encrypted-PDF rule is the next step for that residual.
- **Signature coverage.** Detection of malicious PDFs depends on the rule set and the structural checks; novel indicators for which no rule exists account for the false negatives and are reduced only by growing the rules and the corpus.
- **Sample availability.** Some older real polyglot hashes are unavailable through the single sanctioned download route, bounding the size of the real-sample portion of the adversarial corpus.
- **Inflation depth is bounded but not fatal.** The nested-filter defence peels a configured number of inflation passes — four in the evaluated configuration — and, on reaching that bound, scans what has been surfaced rather than refusing the file. A payload concealed behind a longer filter chain than the configured depth is therefore not exposed, and the file is not refused for having exceeded the bound. This is a deliberate trade against false positives on

legitimately nested documents, but it is an asymmetry in the otherwise fail-closed posture and it is stated here rather than implied: raising the depth narrows the gap at a linear cost, and refusing on depth exhaustion would close it at an unmeasured cost in false positives.

- **Rules-fingerprint strength.** The cache binds a stored verdict to the rule set through the size and last-write timestamp of the rules file rather than through a digest of its contents. This is sufficient for ordinary editing, which changes both, and it costs two filesystem calls per scan instead of hashing the rule set repeatedly. It is nonetheless a weaker binding than a content digest: a modification that preserved both the length and the timestamp — restoring a file from a backup with timestamps retained, for instance — would not invalidate the cached verdicts. Substituting a digest of the rules content, computed once at startup and on file-change notification, closes the gap at negligible cost and is the recommended hardening.
- **A residual type-agreement gap.** Three statements describe an upload’s type — the filename extension, the declared content type, and the bytes — and the pipeline compares only the last two. The extension is checked against an allow-list and searched for a concealed segment, but is never compared against either of the others. A file whose extension disagrees with its content while its declared type agrees with its content therefore satisfies every layer, even though extension-versus-content disagreement is the signature of the documented in-the-wild polyglots. The disagreement is measured and recorded but not made fatal, pending corpus evidence on how frequently benign uploads are simply misnamed.
- **Link inspection does not reach compressed content.** The embedded-link layer extracts and inspects references from the raw content, so a reference inside a compressed document stream is not extracted; the presence of compressed streams is recorded alongside the result so that a clean finding is never mistaken for a complete one. Nor are references decoded before matching, so a scheme expressed through character references or percent-encoding in a vector document — which a browser would decode — is not matched. Both are stated as disclosed blind spots rather than left to be discovered.
- **External references are recorded, not refused.** In the evaluated configuration the layer refuses only dangerous schemes and executable inline-data references; ordinary external references are recorded and permitted, because refusing every external reference carries a benign-collision rate that a deployment should choose rather than inherit. The policy is

exposed as configuration and its cost is reported in the evidence of every scan so that it remains visible in the results.

6.9 Threats to Validity

Following standard practice, threats to validity are separated by type and each is paired with the mitigation applied, as summarized in Table 21.

Type	Threat	Mitigation
Internal	Host AV interfered with detection measurement	Excluded temp directory; RTP otherwise on
Construct	Adversarial rates derived from documented scanner logic	Framed as evasion-resistance, not a population estimate
External	Aggregate metrics from a single PDF corpus	Adversarial corpus reported alongside the aggregate one
External	Some real samples unavailable	Logged as a data-availability limitation
Internal	The gated false-positive rate is derived from recorded severities, not produced by the implemented pipeline	Stated explicitly at the point of reporting; both the ungated and gated rates are given, and the recorded severities are published with the results so the derivation is reproducible
Internal	A crashed scanning engine could be counted as a detection and inflate recall	Engine failures are marked unknown in the evidence and isolated in an <code>is_scanner_error</code> column; findings sourced only from a rule marked test are isolated in <code>is_test_only</code> and excluded from detection figures

Table 21: Threats to validity and the mitigation applied to each.

The construct threat deserves emphasis: the adversarial detection rates characterize behavior on crafted fixtures under each scanner's documented logic, and are best read as an evasion-resistance argument rather than an estimate over a population of real uploads. A second construct threat concerns what a refusal in that comparison demonstrates: a reference validator that refuses a fixture because its type or size is outside what it accepts has shown no inspection capability at all, so such refusals are marked separately in Figure 11 and excluded from the rates in Figure 12 rather than credited as detections. Reporting the aggregate corpus alongside them is what keeps the overall evaluation grounded in realistic files as well as in engineered ones.

6.10 Deployment Considerations

Several properties of the pipeline bear on how it would be deployed in a real system. Because it is training-free and deterministic, it requires no model to maintain, retrain, or version; the only artifact that evolves is the YARA rule set, and the cache is designed to track that evolution automatically. Because it fails closed and enforces hard resource limits, it can be placed directly in the request path without exposing the host to unbounded work from a hostile upload, though a deployment expecting large files should tune the size and expansion limits to its own hardware.

The choice between rejection and neutralization is itself a deployment decision. A high-security context may prefer to reject any file that triggers a transformation, treating neutralization as a signal of suspicion rather than a remedy; a usability-sensitive context may prefer to neutralize wherever possible so that legitimate users are not turned away. Because the verdict and its acting layer are always recorded, either policy can be implemented on top of the same pipeline, and the decision can be revisited without changing the inspection logic. The one non-negotiable operational step is disabling the demo switch before deployment, so that the comparison endpoints used for evaluation are not exposed in production.

6.11 Ethical Considerations

Working with real malicious samples carries obligations that were treated as part of the methodology rather than as an afterthought. Samples were obtained only through a single sanctioned route and handled only under supervisor approval, within a hardened host configuration, and with automatic submission disabled so that samples were not inadvertently redistributed. The two most dangerous real polyglots were deliberately never downloaded, an inert equivalent being built instead, and that containment decision is documented. The purpose throughout was defensive — to build and evaluate a protection — and the report deliberately describes evasion techniques at the level needed to justify the defenses rather than as a construction manual.

6.12 Chapter Summary

Layering resists evasion because independent properties cannot be forged simultaneously, and ordering makes that resistance both efficient and robust. The six self-bypass findings show the principle applied reflexively to the tool; neutralization is a distinct and measurable capability with clearly-stated limits; the evidence trail makes every verdict auditable; and across six structural properties SecureUploader is uniquely complete among the compared approaches, subject to the

stated limitations, threats to validity, and the ethical constraints under which real malware was handled.

Chapter Seven — Conclusion and Future Work

7.1 Summary of the Work

This project set out to build a file-upload defense that resists the evasion techniques which defeat single-check validators, and to treat neutralization as a measurable outcome alongside detection. The result is SecureUploader, a nine-layer, byte-level, training-free pipeline that rejects, neutralizes, or accepts an untrusted file and records an auditable per-layer trace for every verdict. It reaches 95.48% recall and 93.54% accuracy on the full 10,012-file public corpus and is the only validator among six that rejects or neutralizes every fixture in a hand-built adversarial corpus.

7.2 Revisiting the Research Questions

The five research questions posed in Chapter One can now be answered.

- **RQ1 (evasion resistance).** Yes. Because each layer checks an independent property, the pipeline is the only validator among six that rejects or neutralizes every adversarial fixture, covering polyglots, nested-filter concealment, and active SVG content.
- **RQ2 (aggregate performance).** The pipeline reaches 95.48% recall, 93.05% precision, 93.54% accuracy, and a 94.25% F1 score on the full 10,012-file public corpus, under the severity-gated acceptance policy of Section 5.3.1.
- **RQ3 (comparison).** The comparison is one of coverage rather than of hit rate. On the fixtures every validator accepts by type and size, the strongest reference level re-encodes images and matches this work exactly, which is the theoretically expected result. Outside that subset the reference levels accept no PDF, no SVG, and no WebP at all, so they have no answer of any kind to a payload behind nested filters, to active content in a vector image, or to bytes appended to a WebP container — their refusals there are exclusions on type, not detections. The baseline, which never validates the leading bytes, accepts a renamed payload outright. What distinguishes the layered filter is therefore that every evasion class across the whole accepted type set has a dedicated layer.
- **RQ4 (neutralization).** Neutralization adds a third outcome that preserves legitimate files while removing payloads, measurable as a distinct verdict. Plain re-encoding leaves pixel-level (LSB) steganographic payloads intact on lossless formats, but adding a deterministic least-significant-bit clearing step neutralizes every one of the sixty payload-carrying images in a dedicated evaluation, with the fifteen clean controls accepted, while remaining visually imperceptible (peak signal-to-noise ratio no lower than 49.35 dB, mean 52.99 dB); the

residual limit is that the result is established for the lossless raster pixel domain, with transform-domain steganography left for future work.

- **RQ5 (self-bypass).** Yes. Six weaknesses in the pipeline itself were identified by attacking each layer as an adversary aware of its existence, and all six were fixed: nested-filter evasion of a single-pass inflation; unbounded inflation exposing the filter to a decompression bomb of its own; a host antivirus quarantining a decompressed payload before the signature engine could read it, so genuine detections were recorded as failures; a cached verdict masking the detection of a newly added rule; a vector document with a foreign root element reaching the sanitization layer and being stored to be served as an image; and a substring scheme match producing findings on benign text. Two of the six are measurement faults rather than detection faults, which is itself a result: a filter's reported performance is entangled with the environment it runs in and with the harness that reads it.

7.3 Contributions

- **Integration.** An ordered, byte-level pipeline that composes structural, detection, and transformation layers behind a single abstraction into a defense whose layers must all be satisfied.
- **Evasion resistance.** Explicit hardening against nested-filter PDF concealment, image polyglots, and active SVG content, validated on crafted fixtures and organized around an external evasion taxonomy.
- **Neutralization as a second metric.** Deterministic re-encoding and sanitization treated as a first-class verdict and measured alongside detection, with its limits stated honestly.
- **Empirical evaluation.** Aggregate metrics on a large public corpus and a fixture-level comparison against five reference validators, with false-positive and false-negative analyses.
- **Reflexive security.** Six self-bypass vulnerabilities in the pipeline found and fixed, demonstrating the adversarial mindset applied to the tool itself.

7.4 Immediate Next Steps

- **Implement the severity threshold as an acceptance policy.** The per-finding severities recorded during the corpus run show that refusing only high- and critical-severity findings reduces the clean-set false-positive rate from 96.0% to 8.87% at a cost of 0.3 points of recall (Section 5.3.3). That threshold is currently a re-analysis of the recorded results rather than a code path: the signature layers refuse on any match. Exposing a minimum-severity level as configuration, so the pipeline itself produces the gated verdicts, is the first change indicated by the evaluation. Downgrading the encrypted-PDF rule, which dominates the residual 396, follows from the same data.
- **Complete the adversarial run.** Grow the adversarial corpus toward its target size and run the full comparison through the evaluation endpoint to produce detection, false-positive, neutralization, and timing numbers at scale.
- **Ablation study.** Disable each layer in turn to quantify its marginal contribution to detection and neutralization, turning the qualitative coverage map into a measured one.

7.5 Longer-Term Directions

- **Broaden the format scope.** Extend neutralization to additional formats whose structure can be safely rebuilt, each with its own dedicated layer and the same evasion-resistance discipline.
- **Strengthen anti-steganography.** Investigate re-encoding strategies that further reduce the narrow class of pixel-level payloads able to survive a round-trip.
- **Policy-driven verdicts.** Allow deployments to express their own risk tolerance as configuration, choosing per-format between rejection and neutralization.
- **Complementary learned layer.** Explore adding an optional learned detector as an additional layer for the residual false negatives, without sacrificing the transparency of the byte-level core.

7.6 Reflections and Lessons Learned

Two lessons stand out from the engineering. First, the most valuable defensive insight came from attacking the tool itself: each of the six self-bypass findings improved the design more than any external test did, which argues for making the self-bypass mindset a routine part of building a detector rather than a final check. Second, the measurement environment is part of the experiment: the antivirus race showed that a detector's reported performance can be silently distorted by the

host it runs on, so controlling and documenting that environment is as important as the detector's own logic.

7.7 Concluding Remarks

File upload will remain a high-value target as long as servers must judge attacker-controlled bytes. SecureUploader shows that composing a small number of independent, well-ordered layers — and adding the ability to rewrite a file into an inert form — yields a defense that resists evasion far better than any single check, while remaining transparent about every decision it makes. The gap between a leaderboard classifier and a deployable, auditable, training-free defense is exactly the gap this project set out to close.

References

- [1] OWASP Foundation. Unrestricted File Upload and File Upload Cheat Sheet. OWASP, 2023.
- [2] OWASP Foundation. OWASP Top 10 Web Application Security Risks. OWASP, 2021.
- [3] FileUploadChecker: Server-Side Validation of Uploaded Files. Proceedings of ARES 2022.
- [4] Neef, S., & Oudeh, M. FUEL: A Framework for Evaluating File-Upload Vulnerabilities. DIMVA 2024. arXiv:2405.16619.
- [5] Koch, S., et al. On the Abuse and Detection of Polyglot Files. ACM The Web Conference (WWW) 2025. arXiv:2407.01529.
- [6] Issakhani, M., et al. PDF Malware Detection Based on Stacking Learning (CIC-Evasive-PDFMal2022). ICISSP 2022. DOI:10.5220/0010908400003120.
- [7] Al-Haija, Q. A. Machine-Learning Detection of Malicious PDF Documents. 2022.
- [8] Belkind, A., Dubin, R., & Dvir, A. Open-Set Image Round-Trip Sanitization and Steganographic Survival. ICDR 2023. arXiv:2307.14057.
- [9] Albertini, A. Mitra: A Tool for Generating Polyglot Files. Open-source, 2022.
- [10] Albertini, A. Corkami: Studies of File Formats and Polyglots. Open-source project.
- [11] VirusTotal / YARA. YARA: The Pattern Matching Swiss Knife for Malware Researchers. Documentation, 2024.
- [12] abuse.ch. MalwareBazaar: Malware Sample Sharing Platform. API documentation, 2024.
- [13] ImageMagick / Magick.NET. Magick.NET Documentation. 2024.
- [14] Adobe Systems. PDF Reference, Sixth Edition (ISO 32000-1). 2008.
- [15] W3C. Scalable Vector Graphics (SVG) 1.1 Specification. 2011.
- [16] W3C. XML External Entity (XXE) Processing and Security Considerations. 2023.
- [17] EICAR. Standard Anti-Virus Test File. EICAR, 2006.
- [18] Microsoft. ASP.NET Core Middleware and Request Pipeline. .NET 8 Documentation, 2024.
- [19] Microsoft. .NET 8 Runtime and Language Reference. 2024.
- [20] Damn Vulnerable Web Application (DVWA). File Upload Module Documentation. 2023.
- [21] Kaggle. Natural Images Dataset (P. Roy). 2018.
- [22] MITRE. CWE-434: Unrestricted Upload of File with Dangerous Type. 2023.

[23] MITRE. CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-Site Scripting). 2023.

[24] NIST. National Vulnerability Database (NVD). 2024.

[25] Stevens, D. Analyzing Malicious PDF Documents. Technical notes, 2011.